Deep Reinforcement Learning

Unit 4: Policy Gradient and REINFORCE

Hugging Face Deep RL Course

Complete MDX-to-LaTeX study edition

Source: Hugging Face Deep RL Course (original English text).

Contents

1 Introduction **2**

2 What are the policy-based methods? **3**
2.1 Value-based, Policy-based, and Actor-critic methods . . . 3
2.2 The difference between policy-based and policy-gradient methods . . . 5

3 The advantages and disadvantages of policy-gradient methods **5**
3.1 Advantages . . . 5
3.2 Disadvantages . . . 7

4 Diving deeper into policy-gradient methods **8**
4.1 Getting the big picture . . . 8
4.2 Diving deeper into policy-gradient methods . . . 9
4.3 Gradient Ascent and the Policy-gradient Theorem . . . 11
4.4 The Reinforce algorithm (Monte Carlo Reinforce) . . . 13

5 (Optional) the Policy Gradient Theorem **14**

6 Hands on **16**
6.1 Unit 4: Code your first Deep Reinforcement Learning Algorithm with PyTorch: Reinforce. And test its robustness . . . 17
6.2 Let's code Reinforce algorithm from scratch . . . 18
6.3 First agent: Playing CartPole-v1 . . . 21

7 Quiz **35**
7.1 Quiz solutions and explanations . . . 36

8 Conclusion **37**

9 Glossary **38**

10 Additional Readings **38**
10.1 Introduction to Policy Optimization . . . 38
10.2 Policy Gradient . . . 38
10.3 Implementation . . . 38

1 Introduction

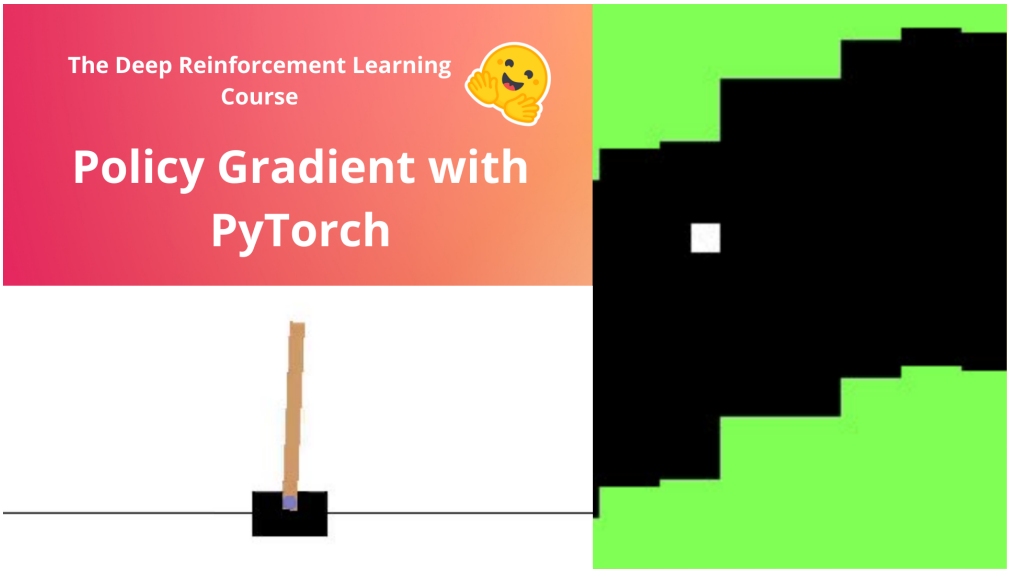


Figure 1: Unit 4: Policy Gradient and REINFORCE course cover [original figure]

In the last unit, we learned about Deep Q-Learning. In this value-based deep reinforcement learning algorithm, we **used a deep neural network to approximate the different Q-values for each possible action at a state.**

Since the beginning of the course, we have only studied value-based methods, **where we estimate a value function as an intermediate step towards finding an optimal policy.**

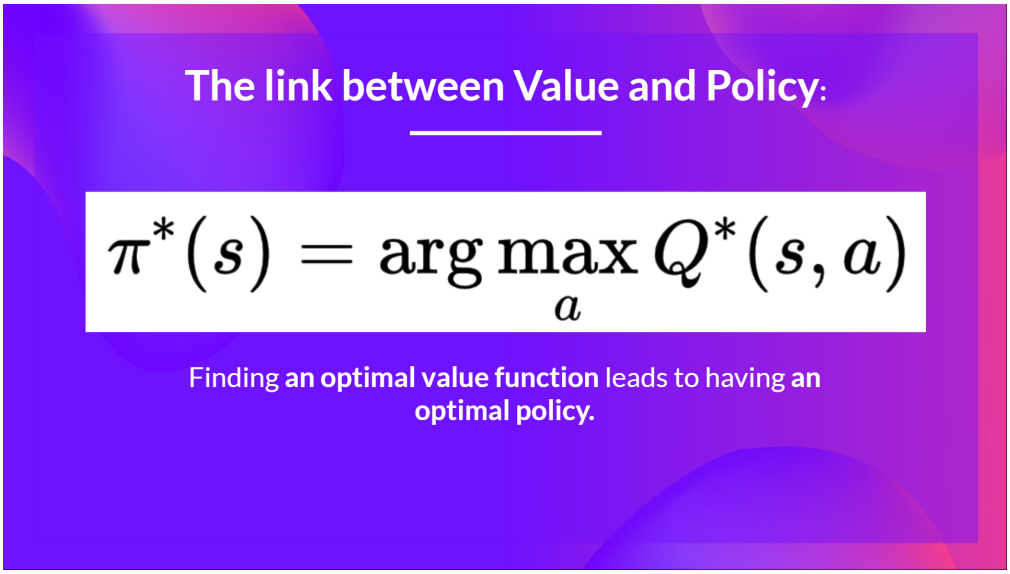


Figure 2: Relation between state-action value estimates and policy selection [original figure]

In value-based methods, the policy π **only exists because of the action value estimates since the policy is just a function** (for instance, greedy-policy) that will select the action with the highest value given a state.

With policy-based methods, we want to optimize the policy directly **without having an intermediate step of learning a value function.**

So today, **we'll learn about policy-based methods and study a subset of these methods called policy gradient**. Then we'll implement our first policy gradient algorithm called Monte Carlo **Reinforce** from scratch using PyTorch. Then, we'll test its robustness using the CartPole-v1 and PixelCopter environments.

You'll then be able to iterate and improve this implementation for more advanced environments.

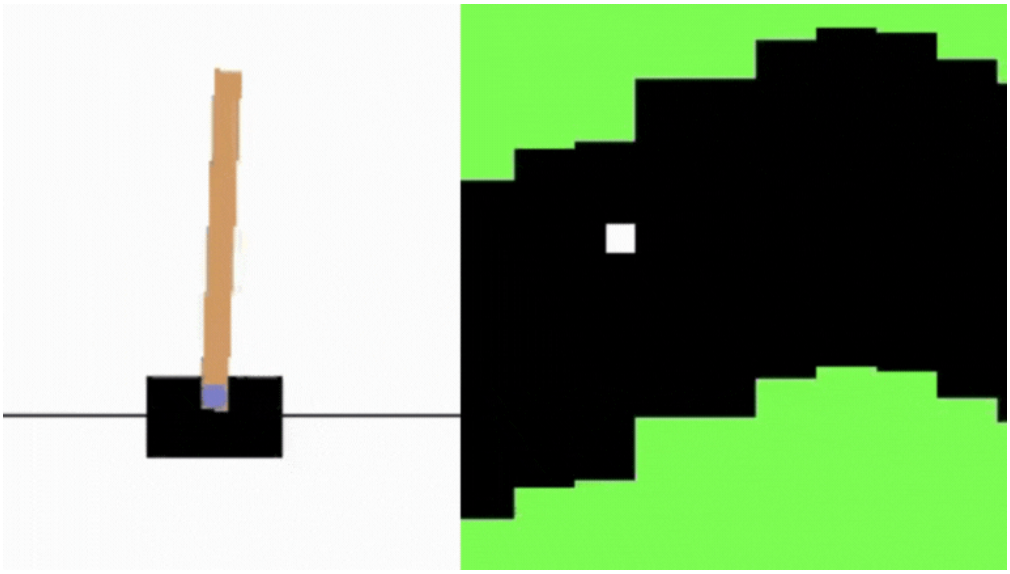


Figure 3: CartPole and PixelCopter example environments (animation still) [original animation]

Let's get started!

2 What are the policy-based methods?

The main goal of Reinforcement learning is to **find the optimal policy π^* that will maximize the expected cumulative reward**. Because Reinforcement Learning is based on the *reward hypothesis*: **all goals can be described as the maximization of the expected cumulative reward.**

For instance, in a soccer game (where you're going to train the agents in two units), the goal is to win the game. We can describe this goal in reinforcement learning as **maximizing the number of goals scored** (when the ball crosses the goal line) into your opponent's soccer goals. And **minimizing the number of goals in your soccer goals**.

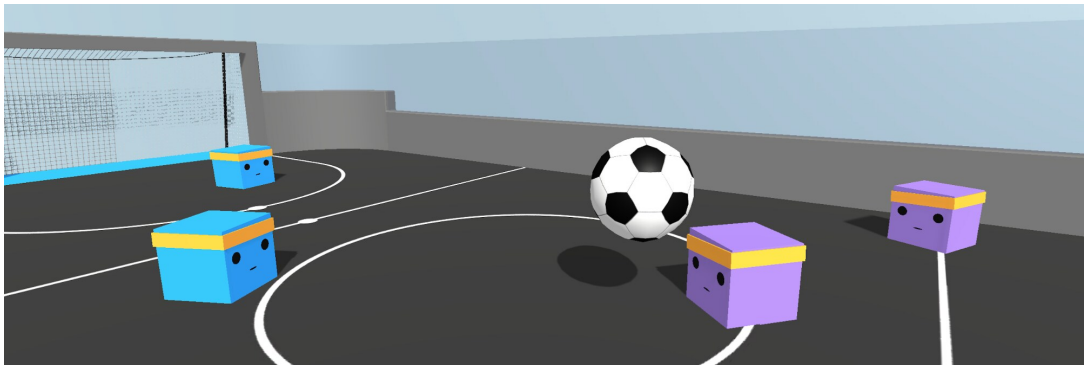


Figure 4: Soccer example for the reward-maximization objective [original figure]

2.1 Value-based, Policy-based, and Actor-critic methods

In the first unit, we saw two methods to find (or, most of the time, approximate) this optimal policy π^*.

- In *value-based methods*, we learn a value function.
  - The idea is that an optimal value function leads to an optimal policy π^*.
  - Our objective is to **minimize the loss between the predicted and target value** to approximate the true action-value function.

  - We have a policy, but it's implicit since it **is generated directly from the value function**. For instance, in Q-Learning, we used an (epsilon-)greedy policy.

- On the other hand, in *policy-based methods*, we directly learn to approximate π^* without having to learn a value function.
  - The idea is **to parameterize the policy**. For instance, using a neural network π_θ, this policy will output a probability distribution over actions (stochastic policy).
  -

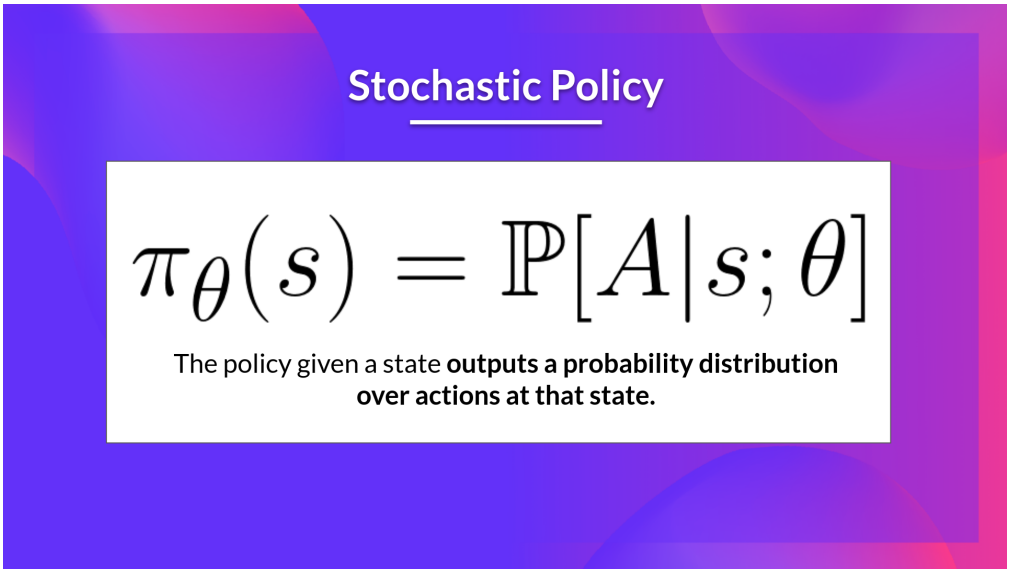


Figure 5: Stochastic policy produces a distribution over actions [original figure]

- Our objective then is **to maximize the performance of the parameterized policy using gradient ascent**.
- To do that, we control the parameter θ that will affect the distribution of actions over a state.

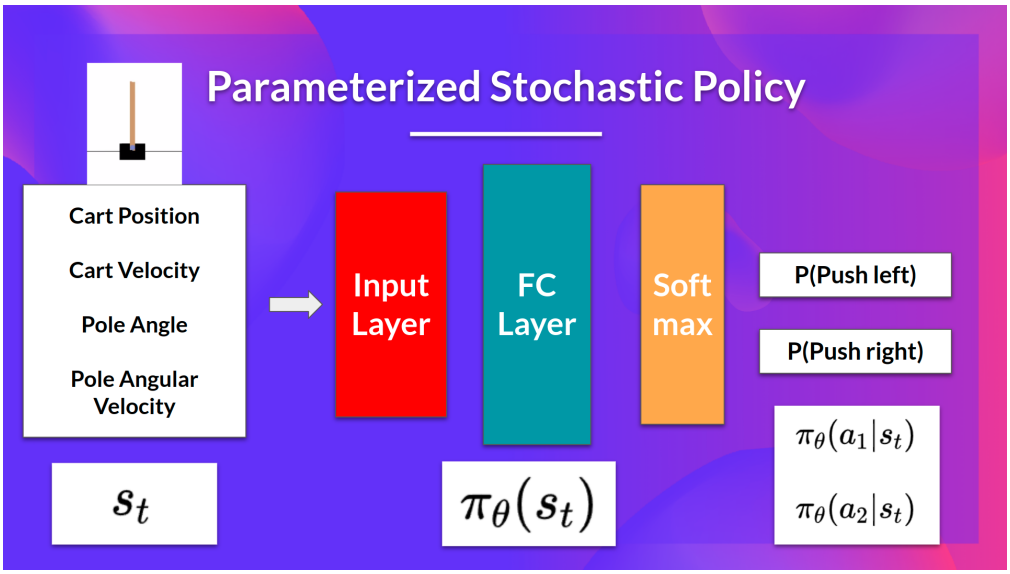


Figure 6: Policy network mapping states to action probabilities [original figure]

- Next time, we'll study the *actor-critic* method, which is a combination of value-based and policy-based methods.

Consequently, thanks to policy-based methods, we can directly optimize our policy π_θ to output a probability distribution over actions $\pi_\theta(a|s)$ that leads to the best cumulative return. To do that,

we define an objective function $J(\theta)$, that is, the expected cumulative reward, and we **want to find the value θ that maximizes this objective function**.

2.2 The difference between policy-based and policy-gradient methods

Policy-gradient methods, what we're going to study in this unit, is a subclass of policy-based methods. In policy-based methods, the optimization is most of the time *on-policy* since for each update, we only use data (trajectories) collected **by our most recent version of π_θ**.

The difference between these two methods **lies on how we optimize the parameter θ**:

- In *policy-based methods*, we search directly for the optimal policy. We can optimize the parameter θ **indirectly** by maximizing the local approximation of the objective function with techniques like hill climbing, simulated annealing, or evolution strategies.
- In *policy-gradient methods*, because it is a subclass of the policy-based methods, we search directly for the optimal policy. But we optimize the parameter θ **directly** by performing the gradient ascent on the performance of the objective function $J(\theta)$.

Before diving more into how policy-gradient methods work (the objective function, policy gradient theorem, gradient ascent, etc.), let's study the advantages and disadvantages of policy-based methods.

3 The advantages and disadvantages of policy-gradient methods

At this point, you might ask, "but Deep Q-Learning is excellent! Why use policy-gradient methods?". To answer this question, let's study the **advantages and disadvantages of policy-gradient methods**.

3.1 Advantages

There are multiple advantages over value-based methods. Let's see some of them:

The simplicity of integration

We can estimate the policy directly without storing additional data (action values).

Policy-gradient methods can learn a stochastic policy

Policy-gradient methods can **learn a stochastic policy while value functions can't**.

This has two consequences:

1. We **don't need to implement an exploration/exploitation trade-off by hand**. Since we output a probability distribution over actions, the agent explores **the state space without always taking the same trajectory.**
2. We also get rid of the problem of **perceptual aliasing**. Perceptual aliasing is when two states seem (or are) the same but need different actions.

Let's take an example: we have an intelligent vacuum cleaner whose goal is to suck the dust and avoid killing the hamsters.

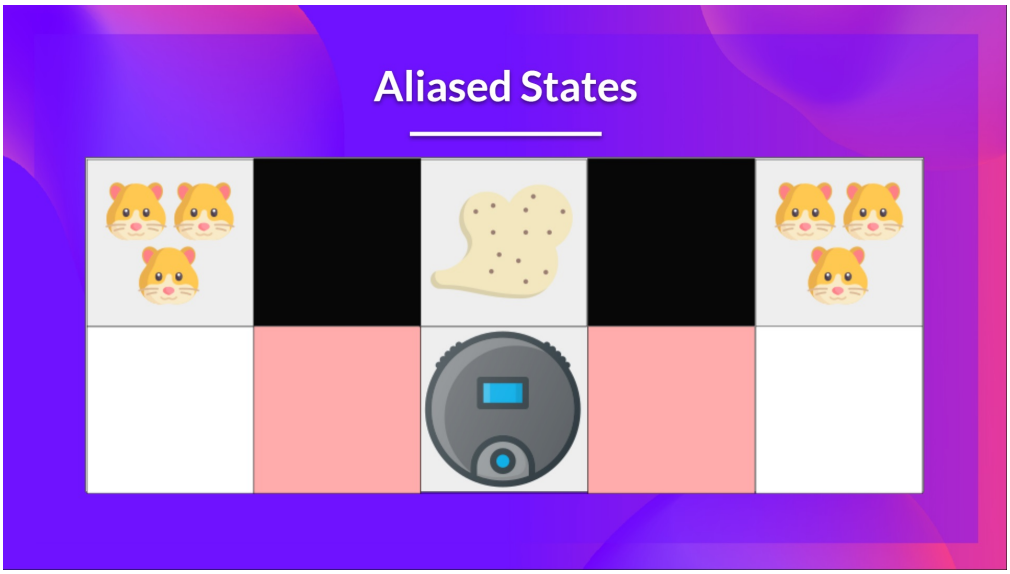


Figure 7: Stochastic policies: first illustration [original figure]

Our vacuum cleaner can only perceive where the walls are.

The problem is that the **two red (colored) states are aliased states because the agent perceives an upper and lower wall for each**.

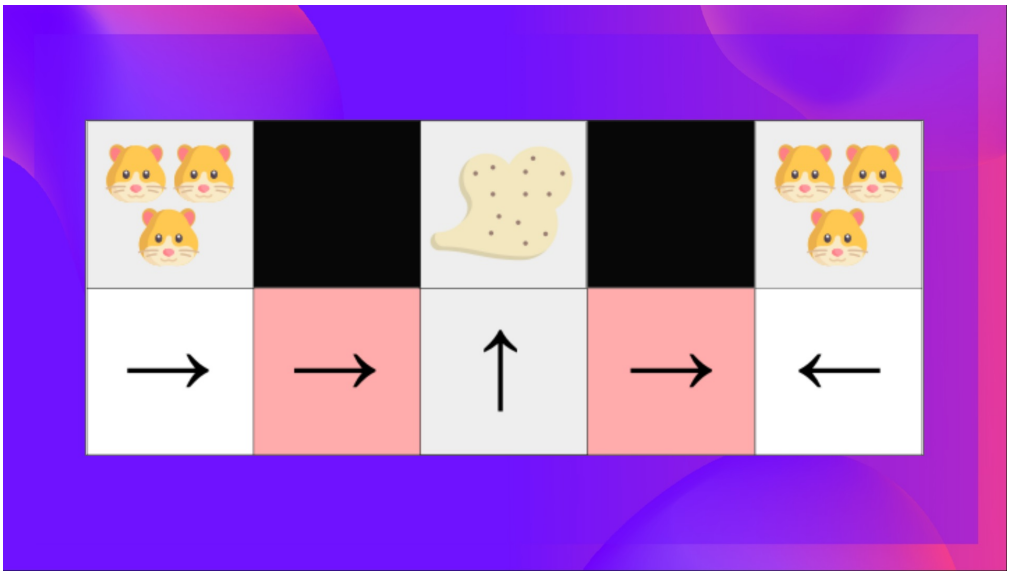


Figure 8: Stochastic policies: second illustration [original figure]

Under a deterministic policy, the policy will either always move right when in a red state or always move left. **Either case will cause our agent to get stuck and never suck the dust**.

Under a value-based Reinforcement learning algorithm, we learn a **quasi-deterministic policy** ("greedy epsilon strategy"). Consequently, our agent can **spend a lot of time before finding the dust**.

On the other hand, an optimal stochastic policy **will randomly move left or right in red (colored) states**. Consequently, **it will not be stuck and will reach the goal state with a high probability**.

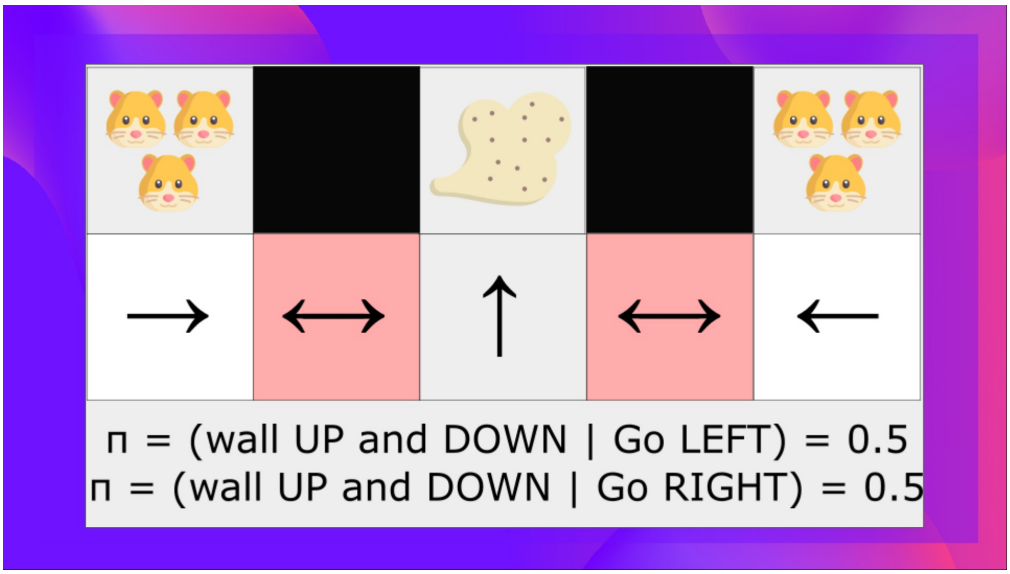


Figure 9: Stochastic policies: third illustration [original figure]

Policy-gradient methods are more effective in high-dimensional action spaces and continuous actions spaces

The problem with Deep Q-learning is that their **predictions assign a score (maximum expected future reward) for each possible action**, at each time step, given the current state.

But what if we have an infinite possibility of actions?

For instance, with a self-driving car, at each state, you can have a (near) infinite choice of actions (turning the wheel at 15°, 17.2°, 19,4°, honking, etc.). **We'll need to output a Q-value for each possible action**! And **taking the max action of a continuous output is an optimization problem itself**!

Instead, with policy-gradient methods, we output a **probability distribution over actions.**

Policy-gradient methods have better convergence properties

In value-based methods, we use an aggressive operator to **change the value function: we take the maximum over Q-estimates**. Consequently, the action probabilities may change dramatically for an arbitrarily small change in the estimated action values if that change results in a different action having the maximal value.

For instance, if during the training, the best action was left (with a Q-value of 0.22) and the training step after it's right (since the right Q-value becomes 0.23), we dramatically changed the policy since now the policy will take most of the time right instead of left.

On the other hand, in policy-gradient methods, stochastic policy action preferences (probability of taking action) **change smoothly over time**.

3.2 Disadvantages

Naturally, policy-gradient methods also have some disadvantages:

- **Frequently, policy-gradient methods converges to a local maximum instead of a global optimum.**
- Policy-gradient goes slower, **step by step: it can take longer to train (inefficient).**
- Policy-gradient can have high variance. We'll see in the actor-critic unit why, and how we can solve this problem.

If you want to go deeper into the advantages and disadvantages of policy-gradient methods, you can check this video.

4 Diving deeper into policy-gradient methods

4.1 Getting the big picture

We just learned that policy-gradient methods aim to find parameters θ that **maximize the expected return**.

The idea is that we have a *parameterized stochastic policy*. In our case, a neural network outputs a probability distribution over actions. The probability of taking each action is also called the *action preference*.

If we take the example of CartPole-v1:

- As input, we have a state.
- As output, we have a probability distribution over actions at that state.

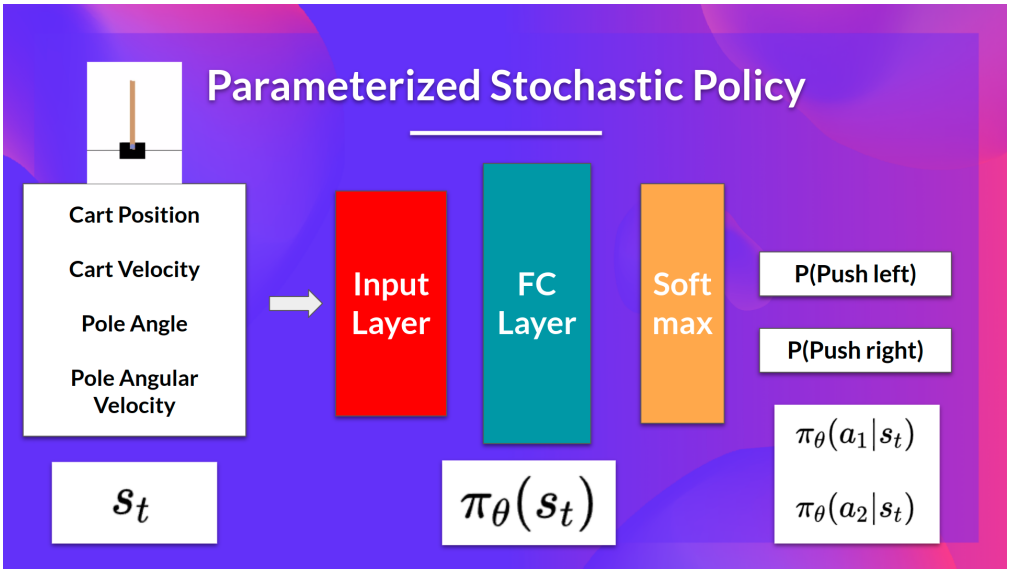


Figure 10: Policy network mapping states to action probabilities [original figure]

Our goal with policy-gradient is to **control the probability distribution of actions** by tuning the policy such that **good actions (that maximize the return) are sampled more frequently in the future.** Each time the agent interacts with the environment, we tweak the parameters such that good actions will be sampled more likely in the future.

But **how are we going to optimize the weights using the expected return**?

The idea is that we're going to **let the agent interact during an episode**. And if we win the episode, we consider that each action taken was good and must be more sampled in the future since they lead to win.

So for each state-action pair, we want to increase the $P(a|s)$: the probability of taking that action at that state. Or decrease if we lost.

The Policy-gradient algorithm (simplified) looks like this:

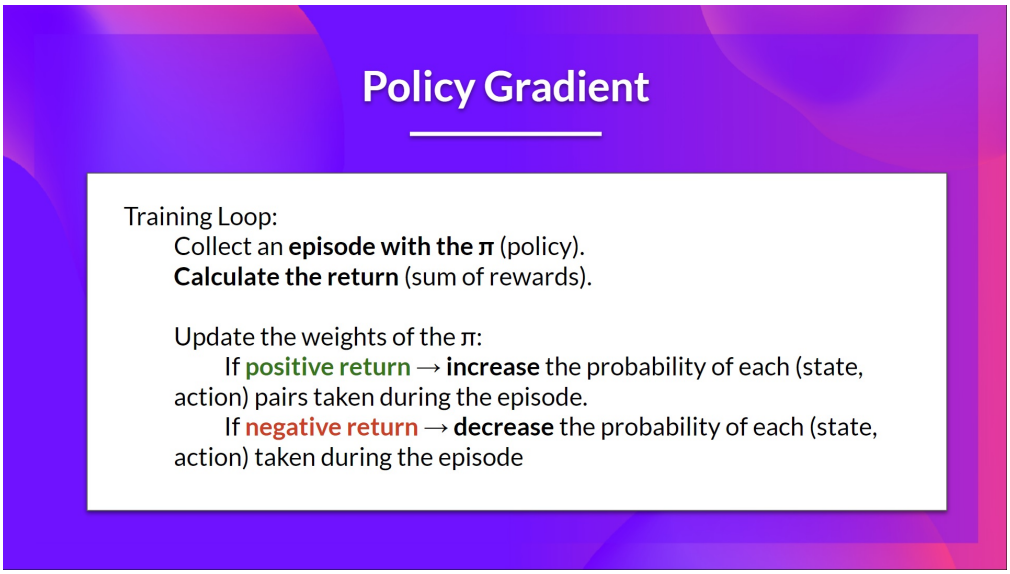


Figure 11: Overview of the policy-gradient update process [original figure]

Now that we got the big picture, let's dive deeper into policy-gradient methods.

4.2 Diving deeper into policy-gradient methods

We have our stochastic policy π which has a parameter θ. This π, given a state, **outputs a probability distribution of actions**.

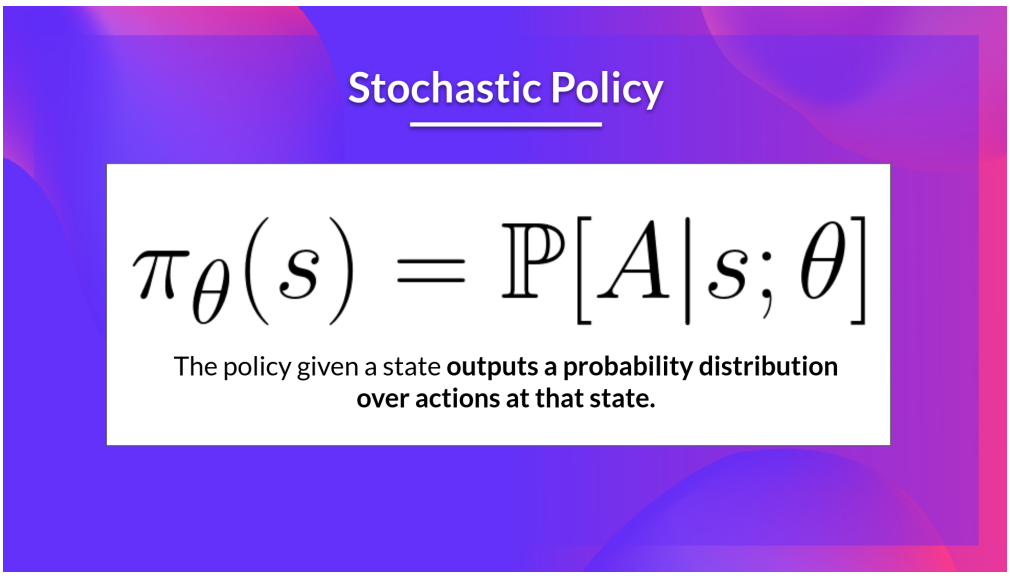


Figure 12: Stochastic policy produces a distribution over actions [original figure]

Where $\pi_\theta(a_t|s_t)$ is the probability of the agent selecting action a_t from state s_t given our policy.

But how do we know if our policy is good? We need to have a way to measure it. To know that, we define a score/objective function called $J(\theta)$.

The objective function

The *objective function* gives us the **performance of the agent** given a trajectory (state action sequence without considering reward (contrary to an episode)), and it outputs the *expected cumulative reward*.

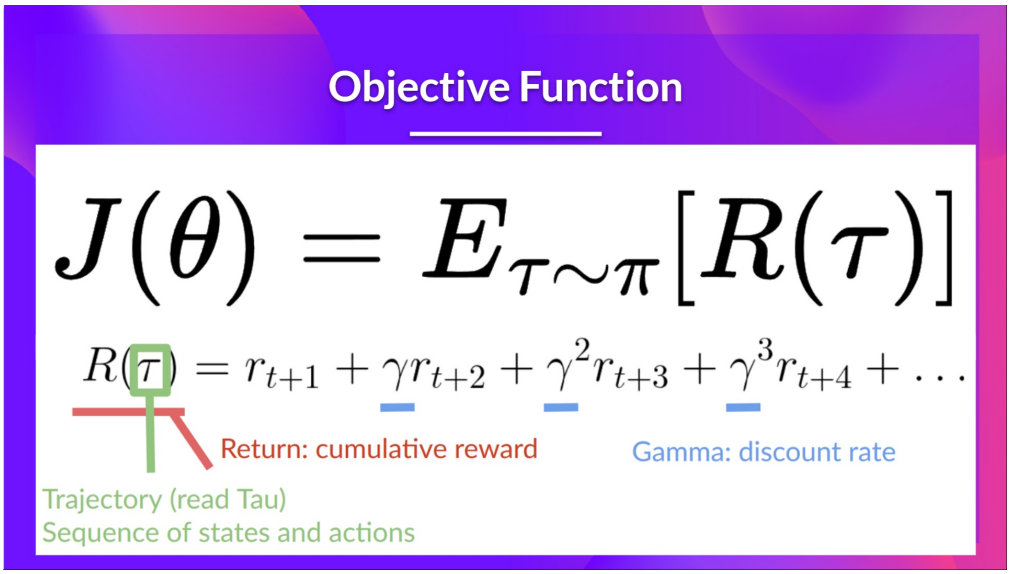


Figure 13: Policy-gradient objective function [original figure]

Let's give some more details on this formula:

- The *expected return* (also called expected cumulative reward), is the weighted average (where the weights are given by $P(\tau;\theta)$ of all possible values that the return $R(\tau)$ can take).

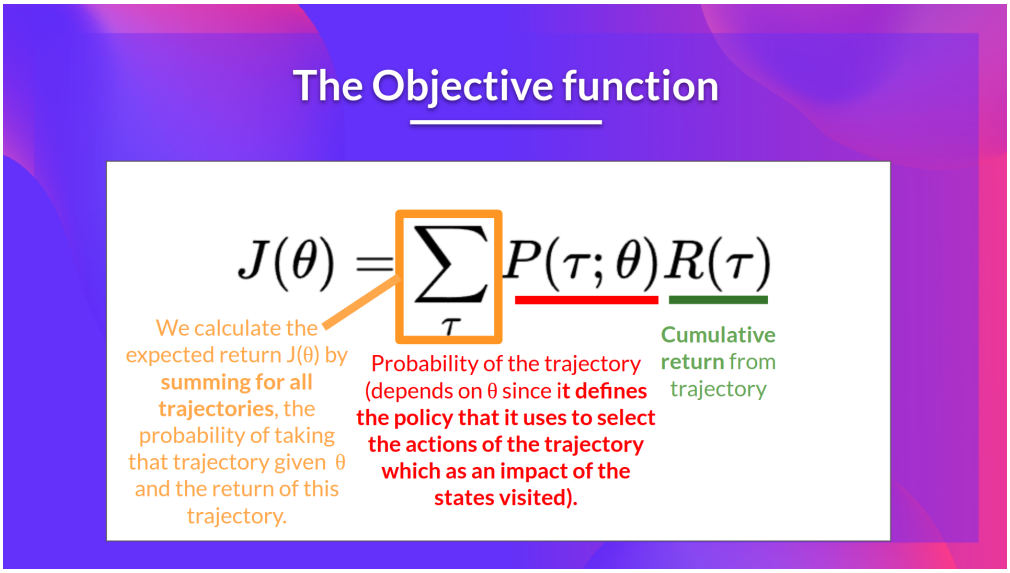


Figure 14: Expected return as the optimization objective [original figure]

- $R(\tau)$: Return from an arbitrary trajectory. To take this quantity and use it to calculate the expected return, we need to multiply it by the probability of each possible trajectory.
- $P(\tau;\theta)$: Probability of each possible trajectory τ (that probability depends on θ since it defines the policy that it uses to select the actions of the trajectory which has an impact of the states visited).

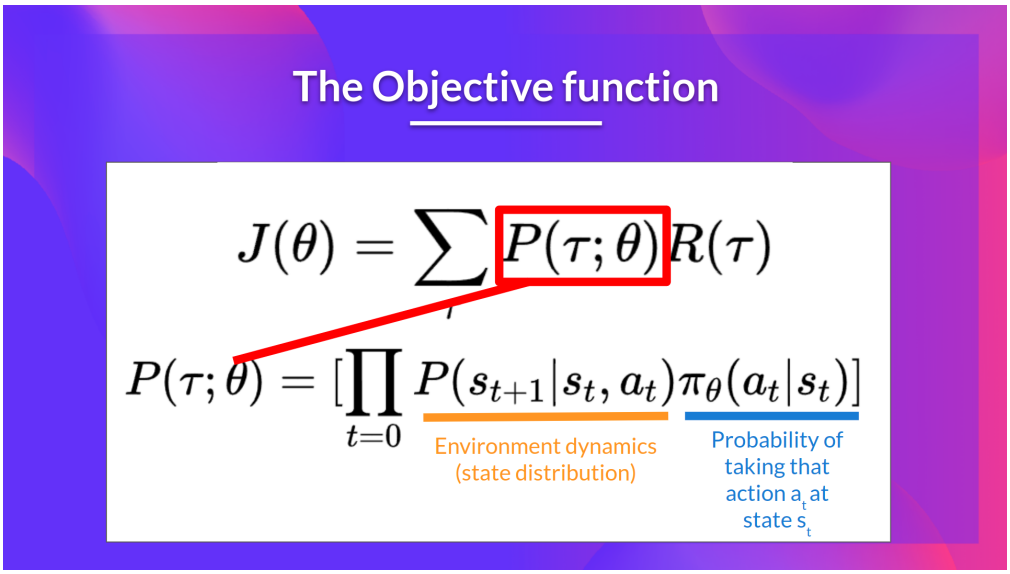


Figure 15: Probability of a trajectory under a parameterized policy [original figure]

- $J(\theta)$: Expected return, we calculate it by summing for all trajectories, the probability of taking that trajectory given θ multiplied by the return of this trajectory.

Our objective then is to maximize the expected cumulative reward by finding the θ that will output the best action probability distributions:

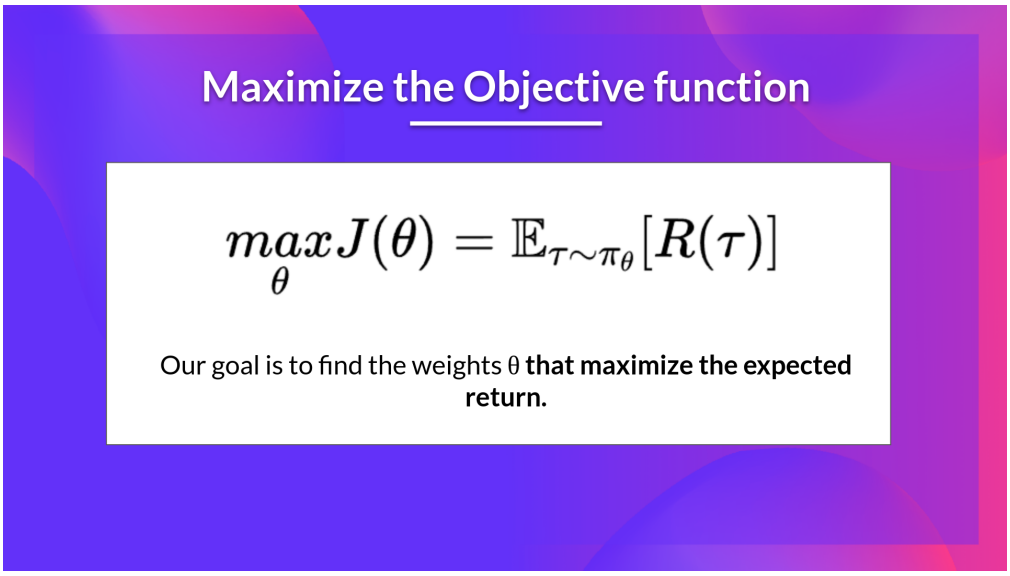


Figure 16: Maximizing the objective function over policy parameters [original figure]

4.3 Gradient Ascent and the Policy-gradient Theorem

Policy-gradient is an optimization problem: we want to find the values of θ that maximize our objective function $J(\theta)$, so we need to use **gradient-ascent**. It's the inverse of *gradient-descent* since it gives the direction of the steepest increase of $J(\theta)$.

(If you need a refresher on the difference between gradient descent and gradient ascent check this and this).

Our update step for gradient-ascent is:

$\theta \leftarrow \theta + \alpha * \nabla_\theta J(\theta)$

We can repeatedly apply this update in the hopes that θ converges to the value that maximizes $J(\theta)$.

However, there are two problems with computing the derivative of $J(\theta)$:

1. We can't calculate the true gradient of the objective function since it requires calculating the probability of each possible trajectory, which is computationally super expensive. So we want to **calculate a gradient estimation with a sample-based estimate (collect some trajectories)**.
2. We have another problem that I explain in the next optional section. To differentiate this objective function, we need to differentiate the state distribution, called the Markov Decision Process dynamics. This is attached to the environment. It gives us the probability of the environment going into the next state, given the current state and the action taken by the agent. The problem is that we can't differentiate it because we might not know about it.

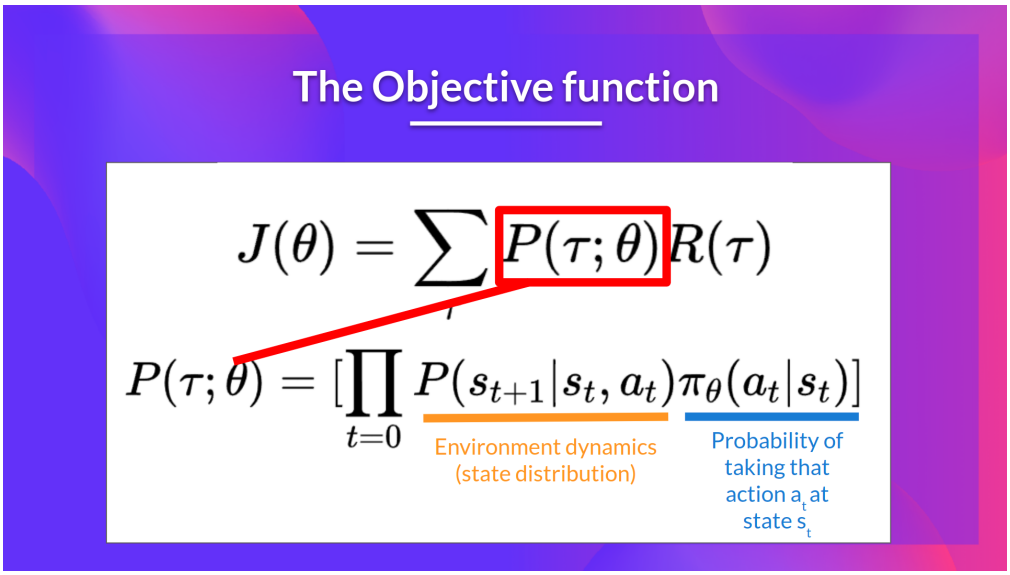


Figure 17: Probability of a trajectory under a parameterized policy [original figure]

Fortunately we're going to use a solution called the Policy Gradient Theorem that will help us to reformulate the objective function into a differentiable function that does not involve the differentiation of the state distribution.

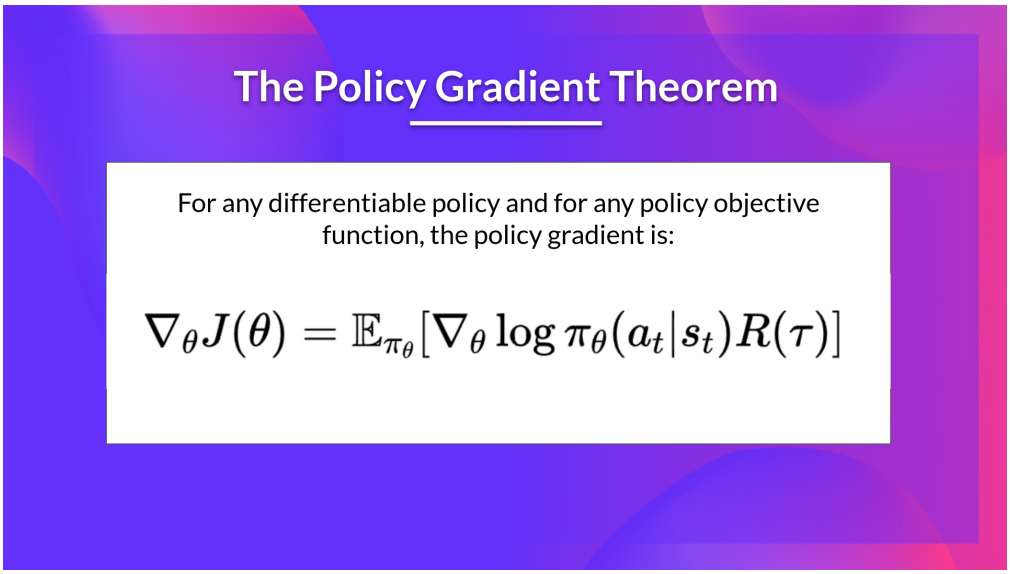


Figure 18: The policy-gradient theorem [original figure]

If you want to understand how we derive this formula for approximating the gradient, check out the next (optional) section.

4.4 The Reinforce algorithm (Monte Carlo Reinforce)

The Reinforce algorithm, also called Monte-Carlo policy-gradient, is a policy-gradient algorithm that **uses an estimated return from an entire episode to update the policy parameter** θ:

In a loop:

- Use the policy π_θ to collect an episode τ
- Use the episode to estimate the gradient $\hat{g} = \nabla_\theta J(\theta)$

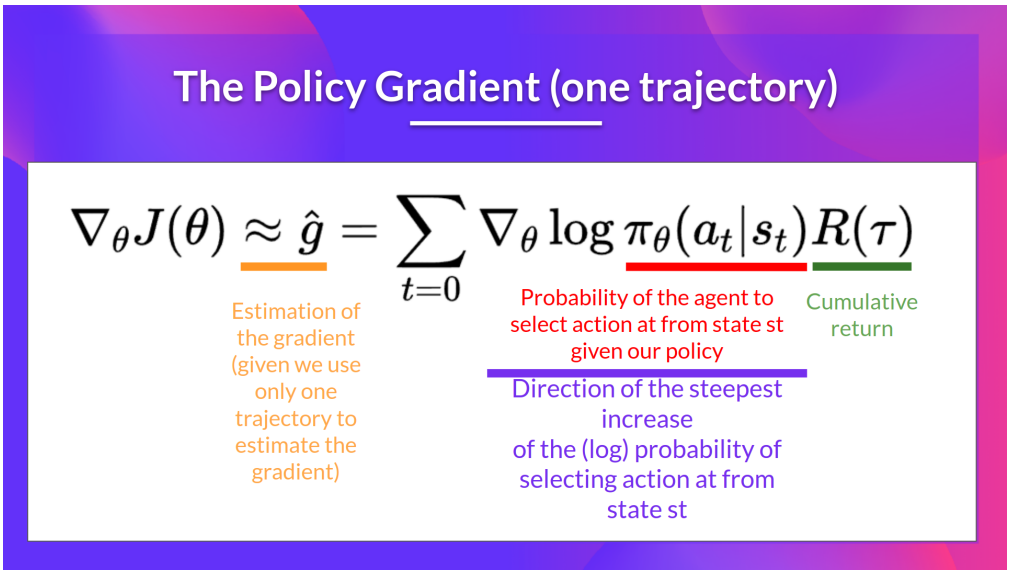


Figure 19: REINFORCE update with a single sampled trajectory [original figure]

- Update the weights of the policy: $\theta \leftarrow \theta + \alpha\hat{g}$

We can interpret this update as follows:

- $\nabla_\theta log \pi_\theta(a_t|s_t)$ is the direction of **steepest increase of the (log) probability** of selecting action a_t from state s_t. This tells us **how we should change the weights of policy** if we want to increase/decrease the log probability of selecting action a_t at state s_t.
- $R(\tau)$: is the scoring function:
  - If the return is high, it will **push up the probabilities** of the (state, action) combinations.
  - Otherwise, if the return is low, it will **push down the probabilities** of the (state, action) combinations.

We can also **collect multiple episodes (trajectories)** to estimate the gradient:

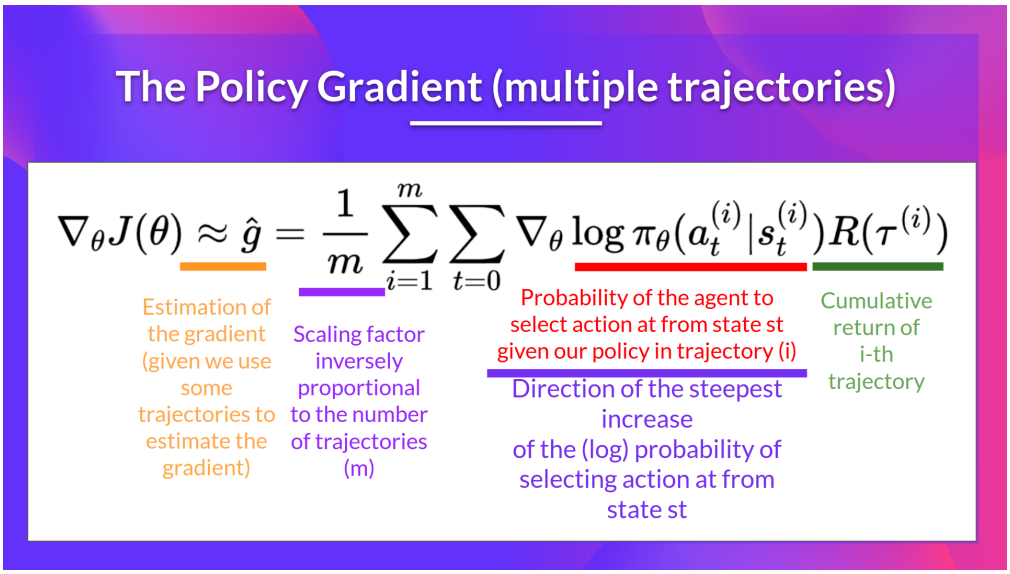


Figure 20: REINFORCE update averaged over multiple trajectories [original figure]

5 (Optional) the Policy Gradient Theorem

In this optional section where we're **going to study how we differentiate the objective function that we will use to approximate the policy gradient**.

Let's first recap our different formulas:

1. The Objective function

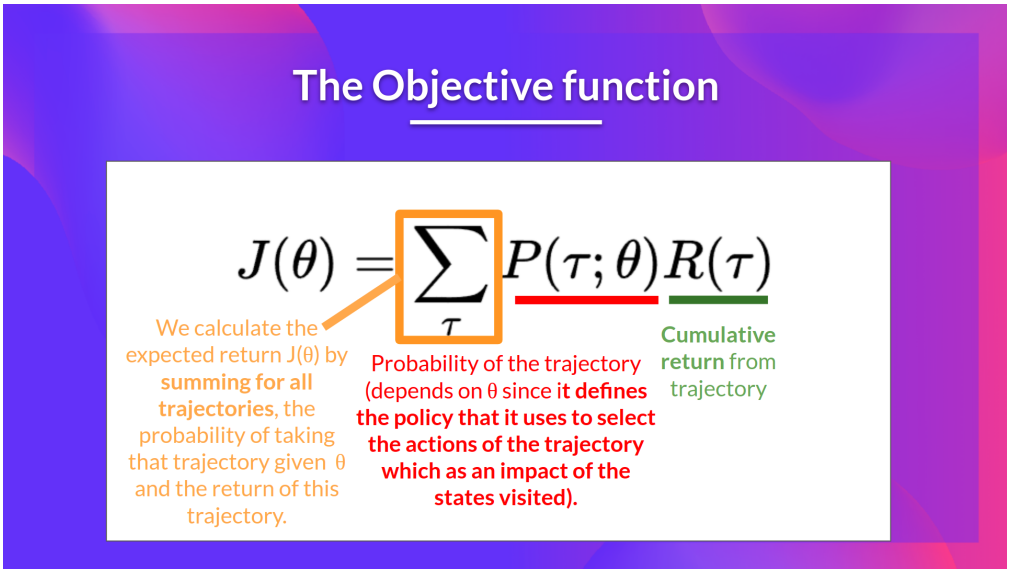


Figure 21: Expected return as the optimization objective [original figure]

2. The probability of a trajectory (given that action comes from π_θ):

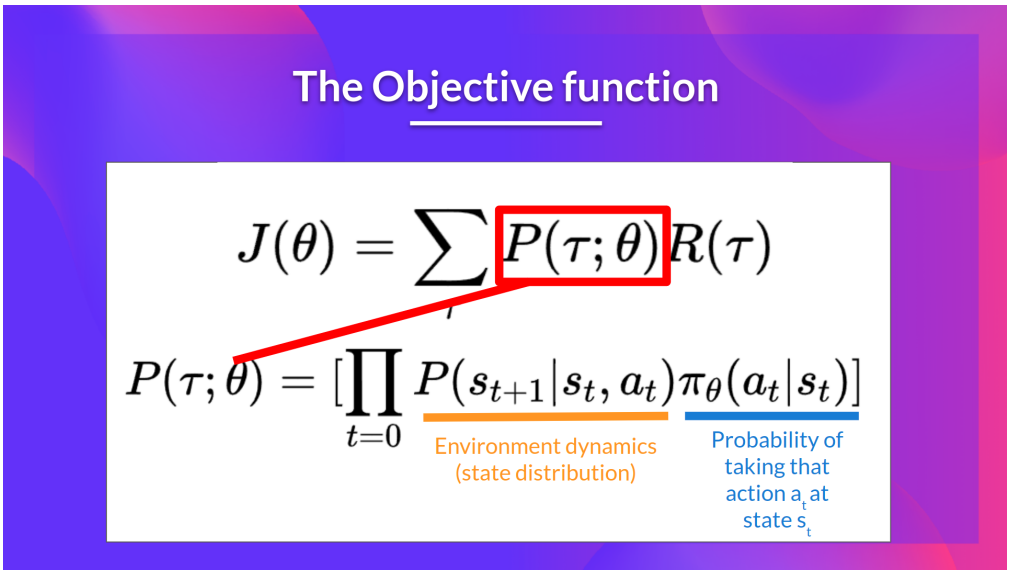


Figure 22: Probability of a trajectory under a parameterized policy [original figure]

So we have:

$\nabla_\theta J(\theta) = \nabla_\theta \sum_\tau P(\tau;\theta)R(\tau)$

We can rewrite the gradient of the sum as the sum of the gradient:

$= \sum_\tau \nabla_\theta (P(\tau;\theta)R(\tau)) = \sum_\tau \nabla_\theta P(\tau;\theta)R(\tau)$ as $R(\tau)$ is not dependent on θ

We then multiply every term in the sum by $\frac{P(\tau;\theta)}{P(\tau;\theta)}$(which is possible since it's $= 1$)

$= \sum_\tau \frac{P(\tau;\theta)}{P(\tau;\theta)} \nabla_\theta P(\tau;\theta)R(\tau)$

We can simplify further this since $\frac{P(\tau;\theta)}{P(\tau;\theta)} \nabla_\theta P(\tau;\theta) = P(\tau;\theta)\frac{\nabla_\theta P(\tau;\theta)}{P(\tau;\theta)}$.

Thus we can rewrite the sum as

$P(\tau;\theta)\frac{\nabla_\theta P(\tau;\theta)}{P(\tau;\theta)} = \sum_\tau P(\tau;\theta)\frac{\nabla_\theta P(\tau;\theta)}{P(\tau;\theta)} R(\tau)$

We can then use the *derivative log trick* (also called *likelihood ratio trick* or *REINFORCE trick*), a simple rule in calculus that implies that $\nabla_x log f(x) = \frac{\nabla_x f(x)}{f(x)}$

So given we have $\frac{\nabla_\theta P(\tau;\theta)}{P(\tau;\theta)}$ we transform it as $\nabla_\theta log P(\tau|\theta)$

So this is our likelihood policy gradient:

$\nabla_\theta J(\theta) = \sum_\tau P(\tau;\theta)\nabla_\theta log P(\tau;\theta)R(\tau)$

Thanks for this new formula, we can estimate the gradient using trajectory samples (we can approximate the likelihood ratio policy gradient with sample-based estimate if you prefer).

$\nabla_\theta J(\theta) = \frac{1}{m}\sum_{i=1}^{m} \nabla_\theta log P(\tau^{(i)};\theta)R(\tau^{(i)})$ where each $\tau^{(i)}$ is a sampled trajectory.

But we still have some mathematics work to do there: we need to simplify $\nabla_\theta log P(\tau|\theta)$

We know that:

$\nabla_\theta log P(\tau^{(i)};\theta) = \nabla_\theta log[\mu(s_0)\prod_{t=0}^{H} P(s_{t+1}^{(i)}|s_t^{(i)}, a_t^{(i)})\pi_\theta(a_t^{(i)}|s_t^{(i)})]$

Where $\mu(s_0)$ is the initial state distribution and $P(s_{t+1}^{(i)}|s_t^{(i)}, a_t^{(i)})$ is the state transition dynamics of the MDP.

We know that the log of a product is equal to the sum of the logs:

$$\nabla_\theta log P(\tau^{(i)};\theta) = \nabla_\theta \left[log\mu(s_0) + \sum_{t=0}^{H} log P(s_{t+1}^{(i)}|s_t^{(i)}a_t^{(i)}) + \sum_{t=0}^{H} log\pi_\theta(a_t^{(i)}|s_t^{(i)}) \right]$$

We also know that the gradient of the sum is equal to the sum of gradient:

$$\nabla_\theta log P(\tau^{(i)};\theta) = \nabla_\theta log\mu(s_0) + \nabla_\theta \sum_{t=0}^{H} log P(s_{t+1}^{(i)}|s_t^{(i)}a_t^{(i)}) + \nabla_\theta \sum_{t=0}^{H} log\pi_\theta(a_t^{(i)}|s_t^{(i)})$$

Since neither initial state distribution or state transition dynamics of the MDP are dependent of θ, the derivate of both terms are 0. So we can remove them:

Since: $\nabla_\theta \sum_{t=0}^{H} logP(s_{t+1}^{(i)}|s_t^{(i)}a_t^{(i)}) = 0$ and $\nabla_\theta \mu(s_0) = 0$

$\nabla_\theta logP(\tau^{(i)};\theta) = \nabla_\theta \sum_{t=0}^{H} log\pi_\theta(a_t^{(i)}|s_t^{(i)})$

We can rewrite the gradient of the sum as the sum of gradients:

$\nabla_\theta logP(\tau^{(i)};\theta) = \sum_{t=0}^{H} \nabla_\theta log\pi_\theta(a_t^{(i)}|s_t^{(i)})$

So, the final formula for estimating the policy gradient is:

$\nabla_\theta J(\theta) = \hat{g} = \frac{1}{m} \sum_{i=1}^{m} \sum_{t=0}^{H} \nabla_\theta \log \pi_\theta(a_t^{(i)}|s_t^{(i)}) R(\tau^{(i)})$

6 Hands on

Official Unit 4 notebook (Google Colab)

Course help / Discord

Now that we've studied the theory behind Reinforce, **you're ready to code your Reinforce agent with PyTorch**. And you'll test its robustness using CartPole-v1 and PixelCopter,.

You'll then be able to iterate and improve this implementation for more advanced environments.

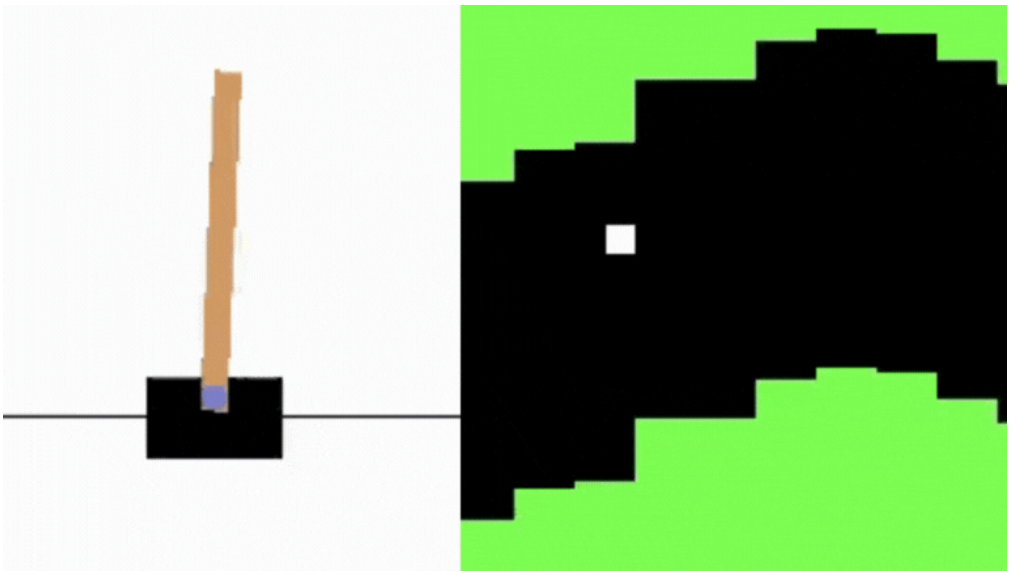


Figure 23: CartPole and PixelCopter example environments (animation still) [original animation]

To validate this hands-on for the certification process, you need to push your trained models to the Hub and:

- Get a result of >= 350 for `Cartpole-v1`
- Get a result of >= 5 for `PixelCopter`.

To find your result, go to the leaderboard and find your model, **the result = mean_reward - std of reward**. **If you don't see your model on the leaderboard, go at the bottom of the leaderboard page and click on the refresh button**.

If you don't find your model, go to the bottom of the page and click on the refresh button.

For more information about the certification process, check this section https://huggingface.co/deep-rl-course/en/unit0/introduction#certification-process

And you can check your progress here https : / / huggingface . co / spaces / ThomasSimonini / Check-my-progress-Deep-RL-Course

To start the hands-on click on Open In Colab button :

Open In Colab

We strongly **recommend students use Google Colab for the hands-on exercises** instead of running them on their personal computers.

By using Google Colab, **you can focus on learning and experimenting without worrying about the technical aspects** of setting up your environments.

6.1 Unit 4: Code your first Deep Reinforcement Learning Algorithm with PyTorch: Reinforce. And test its robustness

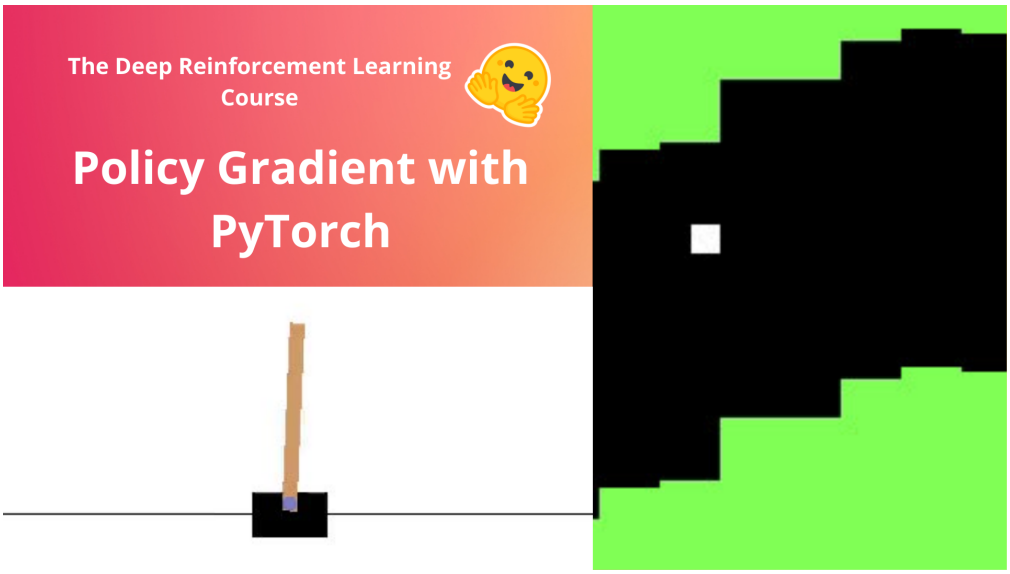


Figure 24: Unit 4: Policy Gradient and REINFORCE course cover [original figure]

In this notebook, you'll code your first Deep Reinforcement Learning algorithm from scratch: Reinforce (also called Monte Carlo Policy Gradient).

Reinforce is a *Policy-based method*: a Deep Reinforcement Learning algorithm that tries **to optimize the policy directly without using an action-value function**.

More precisely, Reinforce is a *Policy-gradient method*, a subclass of *Policy-based methods* that aims **to optimize the policy directly by estimating the weights of the optimal policy using gradient ascent**.

To test its robustness, we're going to train it in 2 different simple environments:

- Cartpole-v1
- PixelcopterEnv

Here is an example of what **you will achieve at the end of this notebook.**

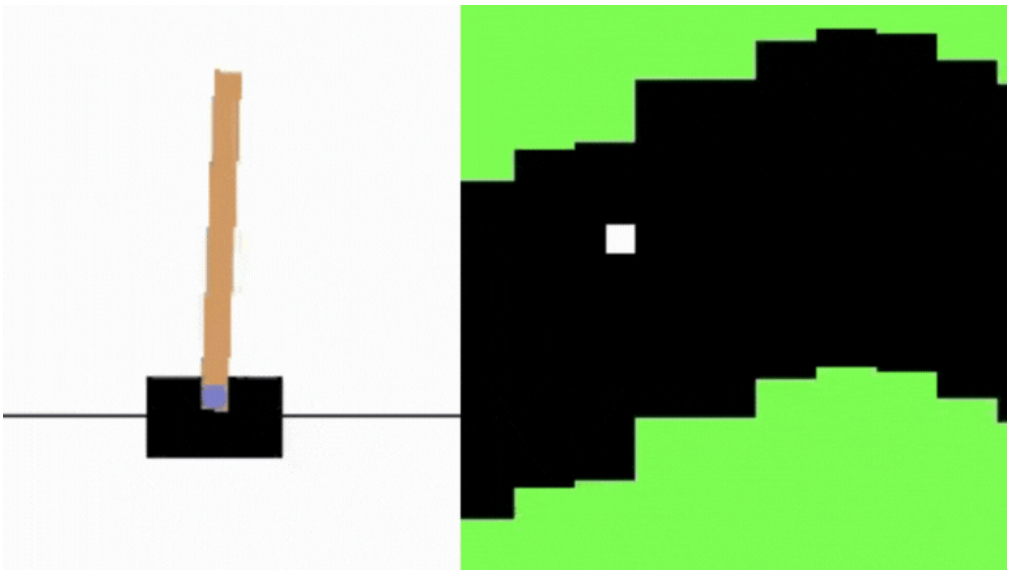


Figure 25: CartPole and PixelCopter example environments (animation still) [original animation]

Environments:

- CartPole-v1
- PixelCopter

RL-Library:

- Python
- PyTorch

We're constantly trying to improve our tutorials, so **if you find some issues in this notebook**, please open an issue on the GitHub Repo.

Objectives of this notebook

At the end of the notebook, you will:

- Be able to **code a Reinforce algorithm from scratch using PyTorch.**
- Be able to **test the robustness of your agent using simple environments.**
- Be able to **push your trained agent to the Hub** with a nice video replay and an evaluation score .

Prerequisites

Before diving into the notebook, you need to:

Study Policy Gradients by reading Unit 4

6.2 Let's code Reinforce algorithm from scratch

Some advice Tip:

It's better to run this colab in a copy on your Google Drive, so that **if it times out** you still have the saved notebook on your Google Drive and do not need to fill everything in from scratch.

To do that you can either do `Ctrl + S` or `File > Save a copy in Google Drive.`

Set the GPU

- To **accelerate the agent's training, we'll use a GPU**. To do that, go to `Runtime > Change Runtime type`

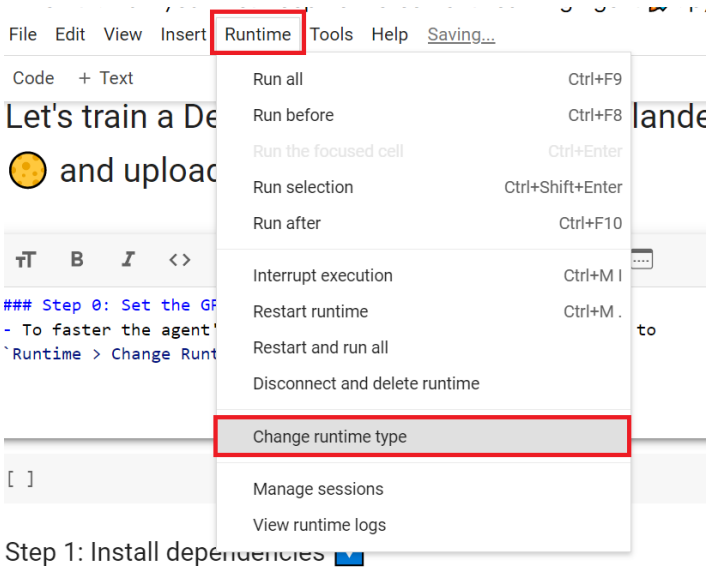


Figure 26: Colab runtime selection: first step [original figure]

- `Hardware Accelerator > GPU`

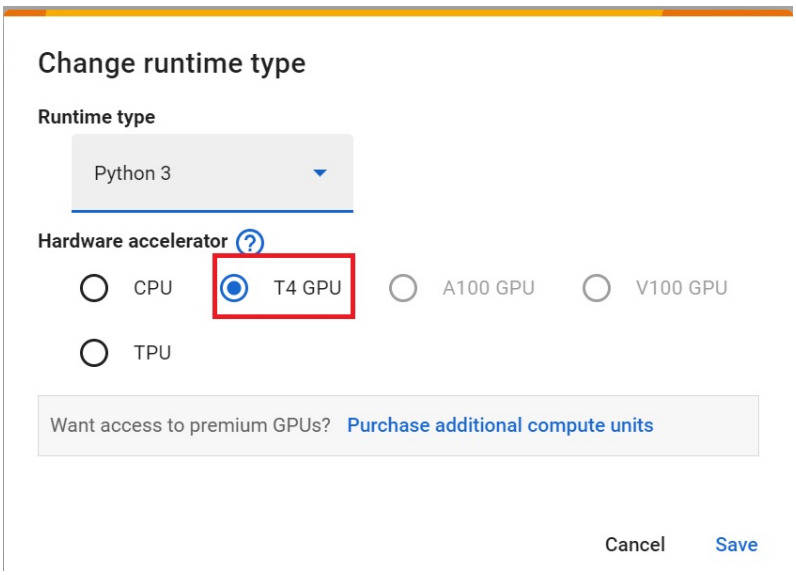


Figure 27: Colab GPU accelerator selection [original figure]

Create a virtual display

During the notebook, we'll need to generate a replay video. To do so, with colab, **we need to have a virtual screen to be able to render the environment** (and thus record the frames).

The following cell will install the librairies and create and run a virtual screen

```
%%capture
!apt install python-opengl
!apt install ffmpeg
!apt install xvfb
!pip install pyvirtualdisplay
!pip install pyglet==1.5.1
```

```
# Virtual display
from pyvirtualdisplay import Display

virtual_display = Display(visible=0, size=(1400, 900))
virtual_display.start()
```

Install the dependencies

The first step is to install the dependencies. We'll install multiple ones:

- `gym`
- `gym-games`: Extra gym environments made with PyGame.
- `huggingface_hub`: The Hub works as a central place where anyone can share and explore models and datasets. It has versioning, metrics, visualizations, and other features that will allow you to easily collaborate with others.

You may be wondering why we install gym and not gymnasium, a more recent version of gym? **Because the gym-games we are using are not updated yet with gymnasium**.

The differences you'll encounter here:

- In `gym` we don't have `terminated` and `truncated` but only `done`.
- In `gym` using `env.step()` returns `state, reward, done, info`

You can learn more about the differences between Gym and Gymnasium here https://gymnasium.farama.org/content/migration-guide/

You can see here all the Reinforce models available https://huggingface.co/models?other=reinforce

And you can find all the Deep Reinforcement Learning models here https://huggingface.co/models?pipeline_tag=reinforcement-learning

```
!pip install -r https://raw.githubusercontent.com/huggingface/deep-rl-class/main/notebooks/unit4/
    requirements-unit4.txt
```

Import the packages

In addition to importing the installed libraries, we also import:

- `imageio`: A library that will help us to generate a replay video

```
import numpy as np

from collections import deque

import matplotlib.pyplot as plt
%matplotlib inline

# PyTorch
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.distributions import Categorical

# Gym
import gym
import gym_pygame

# Hugging Face Hub
from huggingface_hub import notebook_login # To log to our Hugging Face account to be able to
    upload models to the Hub.
import imageio
```

Check if we have a GPU

- Let's check if we have a GPU
- If it's the case you should see `device:cuda0`

```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
```

```
print(device)
```

We're now ready to implement our Reinforce algorithm

6.3 First agent: Playing CartPole-v1

Create the CartPole environment and understand how it works

The environment

Why do we use a simple environment like CartPole-v1? As explained in Reinforcement Learning Tips and Tricks, when you implement your agent from scratch, you need **to be sure that it works correctly and find bugs with easy environments before going deeper** as finding bugs will be much easier in simple environments.

> Try to have some "sign of life" on toy problems

> Validate the implementation by making it run on harder and harder envs (you can compare results against the RL zoo). You usually need to run hyperparameter optimization for that step.

The CartPole-v1 environment

> A pole is attached by an un-actuated joint to a cart, which moves along a frictionless track. The pendulum is placed upright on the cart and the goal is to balance the pole by applying forces in the left and right direction on the cart.

So, we start with CartPole-v1. The goal is to push the cart left or right **so that the pole stays in the equilibrium.**

The episode ends if:

- The pole Angle is greater than ±12°
- The Cart Position is greater than ±2.4
- The episode length is greater than 500

We get a reward of +1 every timestep that the Pole stays in the equilibrium.

```
env_id = "CartPole-v1"
# Create the env
env = gym.make(env_id)

# Create the evaluation env
eval_env = gym.make(env_id)

# Get the state space and action space
s_size = env.observation_space.shape[0]
a_size = env.action_space.n
```

```
print("_____OBSERVATION SPACE_____ \n")
print("The State Space is: ", s_size)
print("Sample observation", env.observation_space.sample())  # Get a random observation
```

```
print("\n _____ACTION SPACE_____ \n")
print("The Action Space is: ", a_size)
print("Action Space Sample", env.action_space.sample())  # Take a random action
```

Let's build the Reinforce Architecture

This implementation is based on three implementations:

- PyTorch official Reinforcement Learning example
- Udacity Reinforce
- Improvement of the integration by Chris1nexus

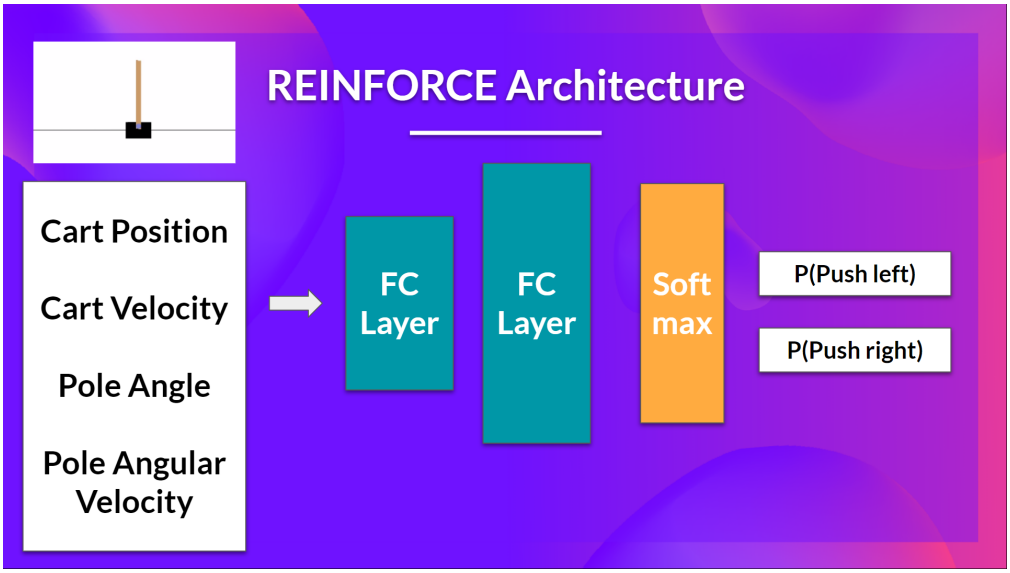


Figure 28: REINFORCE policy-network architecture [original figure]

So we want:

- Two fully connected layers (fc1 and fc2).
- To use ReLU as activation function of fc1
- To use Softmax to output a probability distribution over actions

```
class Policy(nn.Module):
    def __init__(self, s_size, a_size, h_size):
        super(Policy, self).__init__()
        # Create two fully connected layers



    def forward(self, x):
        # Define the forward pass
        # state goes to fc1 then we apply ReLU activation function

        # fc1 outputs goes to fc2

        # We output the softmax

    def act(self, state):
```

```
        """
        Given a state, take action
        """
        state = torch.from_numpy(state).float().unsqueeze(0).to(device)
        probs = self.forward(state).cpu()
        m = Categorical(probs)
        action = np.argmax(m)
        return action.item(), m.log_prob(action)
```

Solution

```
class Policy(nn.Module):
    def __init__(self, s_size, a_size, h_size):
        super(Policy, self).__init__()
        self.fc1 = nn.Linear(s_size, h_size)
        self.fc2 = nn.Linear(h_size, a_size)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return F.softmax(x, dim=1)

    def act(self, state):
        state = torch.from_numpy(state).float().unsqueeze(0).to(device)
        probs = self.forward(state).cpu()
        m = Categorical(probs)
        action = np.argmax(m)
        return action.item(), m.log_prob(action)
```

I made a mistake, can you guess where?

- To find out let's make a forward pass:

```
debug_policy = Policy(s_size, a_size, 64).to(device)
debug_policy.act(env.reset())
```

- Here we see that the error says `ValueError: The value argument to log_prob must be a Tensor`
- It means that `action` in `m.log_prob(action)` must be a Tensor **but it's not.**
- Do you know why? Check the act function and try to see why it does not work.

Advice Tip:: Something is wrong in this implementation. Remember that for the act function **we want to sample an action from the probability distribution over actions**.

(Real) Solution

```
class Policy(nn.Module):
    def __init__(self, s_size, a_size, h_size):
        super(Policy, self).__init__()
        self.fc1 = nn.Linear(s_size, h_size)
        self.fc2 = nn.Linear(h_size, a_size)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return F.softmax(x, dim=1)

    def act(self, state):
        state = torch.from_numpy(state).float().unsqueeze(0).to(device)
```

```
        probs = self.forward(state).cpu()
        m = Categorical(probs)
        action = m.sample()
        return action.item(), m.log_prob(action)
```

By using CartPole, it was easier to debug since **we know that the bug comes from our integration and not from our simple environment**.

- Since **we want to sample an action from the probability distribution over actions**, we can't use `action = np.argmax(m)` since it will always output the action that has the highest probability.
- We need to replace this with `action = m.sample()` which will sample an action from the probability distribution P(.|s)

Let's build the Reinforce Training Algorithm This is the Reinforce algorithm pseudocode:

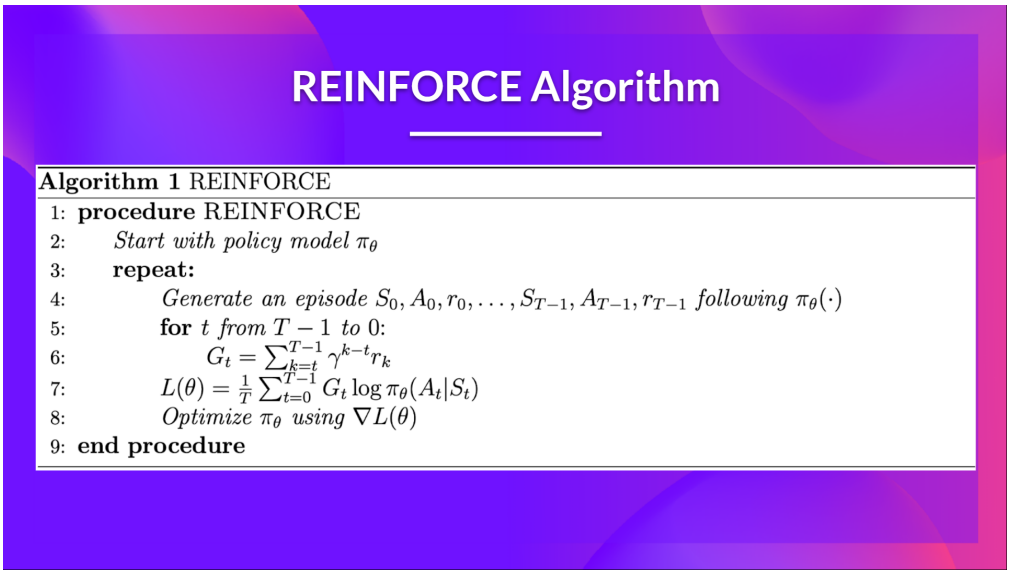


Figure 29: Pseudocode for the REINFORCE algorithm [original figure]

- When we calculate the return Gt (line 6), we see that we calculate the sum of discounted rewards **starting at timestep t**.
- Why? Because our policy should only **reinforce actions on the basis of the consequences**: so rewards obtained before taking an action are useless (since they were not because of the action), **only the ones that come after the action matters**.
- Before coding this you should read this section don't let the past distract you that explains why we use reward-to-go policy gradient.

We use an interesting technique coded by Chris1nexus to **compute the return at each timestep efficiently**. The comments explained the procedure. Don't hesitate also to check the PR explanation But overall the idea is to **compute the return at each timestep efficiently**.

The second question you may ask is **why do we minimize the loss**? Didn't we talk about Gradient Ascent, not Gradient Descent earlier?

- We want to maximize our utility function $J(\theta)$, but in PyTorch and TensorFlow, it's better to **minimize an objective function.**
  - So let's say we want to reinforce action 3 at a certain timestep. Before training this action P is 0.25.

- So we want to modify *theta* such that $\pi_\theta(a_3|s;\theta) > 0.25$
- Because all P must sum to 1, max $pi_\theta(a_3|s;\theta)$ will **minimize other action probability.**
- So we should tell PyTorch **to min** $1 - \pi_\theta(a_3|s;\theta)$**.**
- This loss function approaches 0 as $\pi_\theta(a_3|s;\theta)$ nears 1.
- So we are encouraging the gradient to max $\pi_\theta(a_3|s;\theta)$

```
def reinforce(policy, optimizer, n_training_episodes, max_t, gamma, print_every):
    # Help us to calculate the score during the training
    scores_deque = deque(maxlen=100)
    scores = []
    # Line 3 of pseudocode
    for i_episode in range(1, n_training_episodes+1):
        saved_log_probs = []
        rewards = []
        state = # TODO: reset the environment
        # Line 4 of pseudocode
        for t in range(max_t):
            action, log_prob = # TODO get the action
            saved_log_probs.append(log_prob)
            state, reward, done, _ = # TODO: take an env step
            rewards.append(reward)
            if done:
                break
        scores_deque.append(sum(rewards))
        scores.append(sum(rewards))

        # Line 6 of pseudocode: calculate the return
        returns = deque(maxlen=max_t)
        n_steps = len(rewards)
        # Compute the discounted returns at each timestep,
        # as the sum of the gamma-discounted return at time t (G_t) + the reward at time t

        # In O(N) time, where N is the number of time steps
        # (this definition of the discounted return G_t follows the definition of this quantity
        # shown at page 44 of Sutton&Barto 2017 2nd draft)
        # G_t = r_(t+1) + r_(t+2) + ...

        # Given this formulation, the returns at each timestep t can be computed
        # by re-using the computed future returns G_(t+1) to compute the current return G_t
        # G_t = r_(t+1) + gamma*G_(t+1)
        # G_(t-1) = r_t + gamma* G_t
        # (this follows a dynamic programming approach, with which we memorize solutions in order
        # to avoid computing them multiple times)

        # This is correct since the above is equivalent to (see also page 46 of Sutton&Barto 2017 2
    nd draft)
        # G_(t-1) = r_t + gamma*r_(t+1) + gamma*gamma*r_(t+2) + ...


        ## Given the above, we calculate the returns at timestep t as:
        #               gamma[t] * return[t] + reward[t]
        #
        ## We compute this starting from the last timestep to the first, in order
        ## to employ the formula presented above and avoid redundant computations that would be
    needed
        ## if we were to do it from first to last.

        ## Hence, the queue "returns" will hold the returns in chronological order, from t=0 to t=
    n_steps
        ## thanks to the appendleft() function which allows to append to the position 0 in constant
     time O(1)
        ## a normal python list would instead require O(N) to do this.
        for t in range(n_steps)[::-1]:
```

```
            disc_return_t = (returns[0] if len(returns)>0 else 0)
            returns.appendleft(    ) # TODO: complete here

        ## standardization of the returns is employed to make training more stable
        eps = np.finfo(np.float32).eps.item()

        ## eps is the smallest representable float, which is
        # added to the standard deviation of the returns to avoid numerical instabilities
        returns = torch.tensor(returns)
        returns = (returns - returns.mean()) / (returns.std() + eps)

        # Line 7:
        policy_loss = []
        for log_prob, disc_return in zip(saved_log_probs, returns):
            policy_loss.append(-log_prob * disc_return)
        policy_loss = torch.cat(policy_loss).sum()

        # Line 8: PyTorch prefers gradient descent
        optimizer.zero_grad()
        policy_loss.backward()
        optimizer.step()

        if i_episode % print_every == 0:
            print('Episode {}\tAverage Score: {:.2f}'.format(i_episode, np.mean(scores_deque)))

    return scores
```

Solution

```
def reinforce(policy, optimizer, n_training_episodes, max_t, gamma, print_every):
    # Help us to calculate the score during the training
    scores_deque = deque(maxlen=100)
    scores = []
    # Line 3 of pseudocode
    for i_episode in range(1, n_training_episodes + 1):
        saved_log_probs = []
        rewards = []
        state = env.reset()
        # Line 4 of pseudocode
        for t in range(max_t):
            action, log_prob = policy.act(state)
            saved_log_probs.append(log_prob)
            state, reward, done, _ = env.step(action)
            rewards.append(reward)
            if done:
                break
        scores_deque.append(sum(rewards))
        scores.append(sum(rewards))

        # Line 6 of pseudocode: calculate the return
        returns = deque(maxlen=max_t)
        n_steps = len(rewards)
        # Compute the discounted returns at each timestep,
        # as
        #      the sum of the gamma-discounted return at time t (G_t) + the reward at time t
        #
        # In O(N) time, where N is the number of time steps
        # (this definition of the discounted return G_t follows the definition of this quantity
        # shown at page 44 of Sutton&Barto 2017 2nd draft)
        # G_t = r_(t+1) + r_(t+2) + ...

        # Given this formulation, the returns at each timestep t can be computed
        # by re-using the computed future returns G_(t+1) to compute the current return G_t
        # G_t = r_(t+1) + gamma*G_(t+1)
```

```
        # G_(t-1) = r_t + gamma* G_t
        # (this follows a dynamic programming approach, with which we memorize solutions in order
        # to avoid computing them multiple times)

        # This is correct since the above is equivalent to (see also page 46 of Sutton&Barto 2017 2
    nd draft)
        # G_(t-1) = r_t + gamma*r_(t+1) + gamma*gamma*r_(t+2) + ...

        ## Given the above, we calculate the returns at timestep t as:
        #               gamma[t] * return[t] + reward[t]
        #
        ## We compute this starting from the last timestep to the first, in order
        ## to employ the formula presented above and avoid redundant computations that would be
    needed
        ## if we were to do it from first to last.

        ## Hence, the queue "returns" will hold the returns in chronological order, from t=0 to t=
    n_steps
        ## thanks to the appendleft() function which allows to append to the position 0 in constant
     time O(1)
        ## a normal python list would instead require O(N) to do this.
        for t in range(n_steps)[::-1]:
            disc_return_t = returns[0] if len(returns) > 0 else 0
            returns.appendleft(gamma * disc_return_t + rewards[t])

        ## standardization of the returns is employed to make training more stable
        eps = np.finfo(np.float32).eps.item()
        ## eps is the smallest representable float, which is
        # added to the standard deviation of the returns to avoid numerical instabilities
        returns = torch.tensor(returns)
        returns = (returns - returns.mean()) / (returns.std() + eps)

        # Line 7:
        policy_loss = []
        for log_prob, disc_return in zip(saved_log_probs, returns):
            policy_loss.append(-log_prob * disc_return)
        policy_loss = torch.cat(policy_loss).sum()

        # Line 8: PyTorch prefers gradient descent
        optimizer.zero_grad()
        policy_loss.backward()
        optimizer.step()

        if i_episode % print_every == 0:
            print("Episode {}\tAverage Score: {:.2f}".format(i_episode, np.mean(scores_deque)))

    return scores
```

Train it

- We're now ready to train our agent.
- But first, we define a variable containing all the training hyperparameters.
- You can change the training parameters (and should)

```
cartpole_hyperparameters = {
    "h_size": 16,
    "n_training_episodes": 1000,
    "n_evaluation_episodes": 10,
    "max_t": 1000,
    "gamma": 1.0,
    "lr": 1e-2,
    "env_id": env_id,
    "state_space": s_size,
```

```
    "action_space": a_size,
}
```

```
# Create policy and place it to the device
cartpole_policy = Policy(
    cartpole_hyperparameters["state_space"],
    cartpole_hyperparameters["action_space"],
    cartpole_hyperparameters["h_size"],
).to(device)
cartpole_optimizer = optim.Adam(cartpole_policy.parameters(), lr=cartpole_hyperparameters["lr"])
```

```
scores = reinforce(
    cartpole_policy,
    cartpole_optimizer,
    cartpole_hyperparameters["n_training_episodes"],
    cartpole_hyperparameters["max_t"],
    cartpole_hyperparameters["gamma"],
    100,
)
```

Define evaluation method

- Here we define the evaluation method that we're going to use to test our Reinforce agent.

```
def evaluate_agent(env, max_steps, n_eval_episodes, policy):
    """
    Evaluate the agent for ``n_eval_episodes`` episodes and returns average reward and std of
    reward.
    :param env: The evaluation environment
    :param n_eval_episodes: Number of episode to evaluate the agent
    :param policy: The Reinforce agent
    """
    episode_rewards = []
    for episode in range(n_eval_episodes):
        state = env.reset()
        step = 0
        done = False
        total_rewards_ep = 0

        for step in range(max_steps):
            action, _ = policy.act(state)
            new_state, reward, done, info = env.step(action)
            total_rewards_ep += reward

            if done:
                break
            state = new_state
        episode_rewards.append(total_rewards_ep)
    mean_reward = np.mean(episode_rewards)
    std_reward = np.std(episode_rewards)

    return mean_reward, std_reward
```

Evaluate our agent

```
evaluate_agent(
    eval_env, cartpole_hyperparameters["max_t"], cartpole_hyperparameters["n_evaluation_episodes"],
     cartpole_policy
)
```

Publish our trained model on the Hub Now that we saw we got good results after the training, we can publish our trained model on the hub with one line of code.

Here's an example of a Model Card:

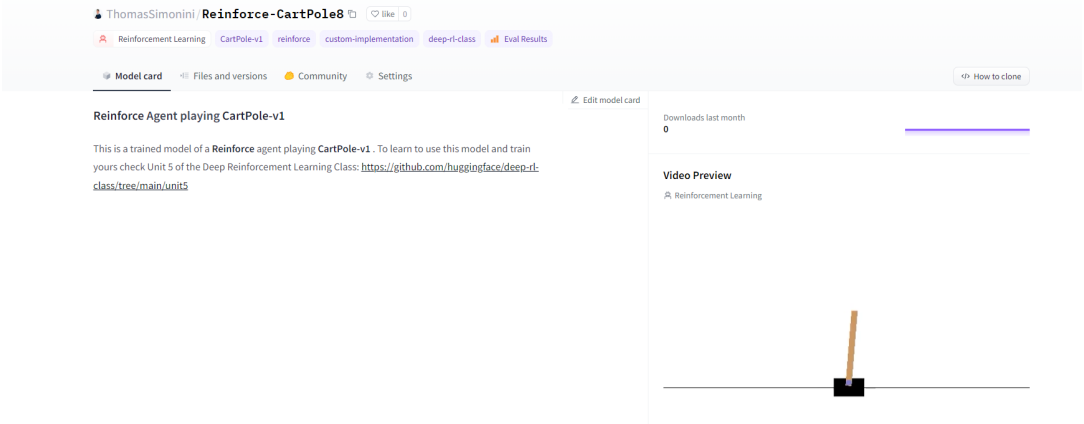


Figure 30: Hugging Face model card interface [original figure]

Push to the Hub

Do not modify this code

```
from huggingface_hub import HfApi, snapshot_download
from huggingface_hub.repocard import metadata_eval_result, metadata_save

from pathlib import Path
import datetime
import json
import imageio

import tempfile

import os
```

```
def record_video(env, policy, out_directory, fps=30):
    """
    Generate a replay video of the agent
    :param env
    :param Qtable: Qtable of our agent
    :param out_directory
    :param fps: how many frame per seconds (with taxi-v3 and frozenlake-v1 we use 1)
    """
    images = []
    done = False
    state = env.reset()
    img = env.render(mode="rgb_array")
    images.append(img)
    while not done:
        # Take the action (index) that have the maximum expected future reward given that state
        action, _ = policy.act(state)
        state, reward, done, info = env.step(action)  # We directly put next_state = state for
    recording logic
        img = env.render(mode="rgb_array")
        images.append(img)
    imageio.mimsave(out_directory, [np.array(img) for i, img in enumerate(images)], fps=fps)
```

```
def push_to_hub(repo_id,
                model,
                hyperparameters,
                eval_env,
                video_fps=30
                ):
  """
  Evaluate, Generate a video and Upload a model to Hugging Face Hub.
  This method does the complete pipeline:
  - It evaluates the model
  - It generates the model card
  - It generates a replay video of the agent
  - It pushes everything to the Hub

  :param repo_id: repo_id: id of the model repository from the Hugging Face Hub
  :param model: the pytorch model we want to save
  :param hyperparameters: training hyperparameters
  :param eval_env: evaluation environment
  :param video_fps: how many frame per seconds to record our video replay
  """

  _, repo_name = repo_id.split("/")
  api = HfApi()

  # Step 1: Create the repo
  repo_url = api.create_repo(
        repo_id=repo_id,
        exist_ok=True,
  )

  with tempfile.TemporaryDirectory() as tmpdirname:
    local_directory = Path(tmpdirname)

    # Step 2: Save the model
    torch.save(model, local_directory / "model.pt")

    # Step 3: Save the hyperparameters to JSON
    with open(local_directory / "hyperparameters.json", "w") as outfile:
      json.dump(hyperparameters, outfile)

    # Step 4: Evaluate the model and build JSON
    mean_reward, std_reward = evaluate_agent(eval_env,
                                             hyperparameters["max_t"],
                                             hyperparameters["n_evaluation_episodes"],
                                             model)
    # Get datetime
    eval_datetime = datetime.datetime.now()
    eval_form_datetime = eval_datetime.isoformat()

    evaluate_data = {
          "env_id": hyperparameters["env_id"],
          "mean_reward": mean_reward,
          "n_evaluation_episodes": hyperparameters["n_evaluation_episodes"],
          "eval_datetime": eval_form_datetime,
    }

    # Write a JSON file
    with open(local_directory / "results.json", "w") as outfile:
        json.dump(evaluate_data, outfile)

    # Step 5: Create the model card
    env_name = hyperparameters["env_id"]

    metadata = {}
```

```
  metadata["tags"] = [
        env_name,
        "reinforce",
        "reinforcement-learning",
        "custom-implementation",
        "deep-rl-class"
    ]

  # Add metrics
  eval = metadata_eval_result(
      model_pretty_name=repo_name,
      task_pretty_name="reinforcement-learning",
      task_id="reinforcement-learning",
      metrics_pretty_name="mean_reward",
      metrics_id="mean_reward",
      metrics_value=f"{mean_reward:.2f} +/- {std_reward:.2f}",
      dataset_pretty_name=env_name,
      dataset_id=env_name,
    )

  # Merges both dictionaries
  metadata = {**metadata, **eval}

  model_card = f"""
# **Reinforce** Agent playing **{env_id}**
This is a trained model of a **Reinforce** agent playing **{env_id}** .
To learn to use this model and train yours check Unit 4 of the Deep Reinforcement Learning Course:
   https://huggingface.co/deep-rl-course/unit4/introduction
"""

  readme_path = local_directory / "README.md"
  readme = ""
  if readme_path.exists():
      with readme_path.open("r", encoding="utf8") as f:
        readme = f.read()
  else:
    readme = model_card

  with readme_path.open("w", encoding="utf-8") as f:
    f.write(readme)

  # Save our metrics to Readme metadata
  metadata_save(readme_path, metadata)

  # Step 6: Record a video
  video_path =  local_directory / "replay.mp4"
  record_video(env, model, video_path, video_fps)

  # Step 7. Push everything to the Hub
  api.upload_folder(
        repo_id=repo_id,
        folder_path=local_directory,
        path_in_repo=".",
  )

  print(f"Your model is pushed to the Hub. You can view your model here: {repo_url}")
```

By using `push_to_hub`, **you evaluate, record a replay, generate a model card of your agent, and push it to the Hub**.

This way:

- You can **showcase our work**
- You can **visualize your agent playing**

- You can **share an agent with the community that others can use**
- You can **access a leaderboard to see how well your agent is performing compared to your classmates** https : / / huggingface . co / spaces / huggingface-projects / Deep-Reinforcement-Learning-Leaderboard

To be able to share your model with the community there are three more steps to follow:

1. (If it's not already done) create an account to HF -> https://huggingface.co/join
2. Sign in and then, you need to store your authentication token from the Hugging Face website.

- Create a new token (https://huggingface.co/settings/tokens) **with write role**

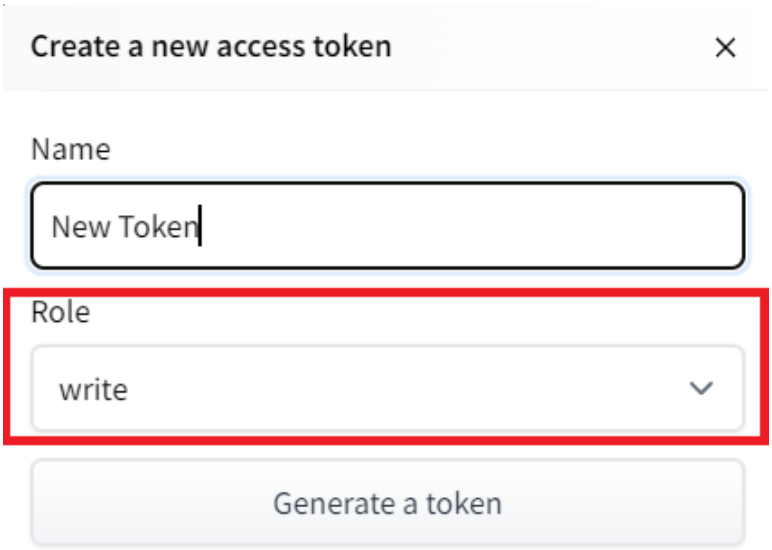


Figure 31: Creating a Hugging Face access token [original figure]

```
notebook_login()
```

If you don't want to use Google Colab or a Jupyter Notebook, you need to use this command instead: `huggingface-cli login` (or `login`)

3. We're now ready to push our trained agent to the Hub using `package_to_hub()` function

```
repo_id = ""  # TODO Define your repo id {username/Reinforce-{model-id}}
push_to_hub(
    repo_id,
    cartpole_policy,  # The model we want to save
    cartpole_hyperparameters,  # Hyperparameters
    eval_env,  # Evaluation environment
    video_fps=30
)
```

Now that we tested the robustness of our implementation, let's try a more complex environment: PixelCopter

Second agent: PixelCopter

Study the PixelCopter environment

- The Environment documentation

```
env_id = "Pixelcopter-PLE-v0"
env = gym.make(env_id)
eval_env = gym.make(env_id)
s_size = env.observation_space.shape[0]
a_size = env.action_space.n
```

```
print("_____OBSERVATION SPACE_____ \n")
print("The State Space is: ", s_size)
print("Sample observation", env.observation_space.sample())  # Get a random observation
```

```
print("\n _____ACTION SPACE_____ \n")
print("The Action Space is: ", a_size)
print("Action Space Sample", env.action_space.sample())  # Take a random action
```

The observation space (7) :

- player y position
- player velocity
- player distance to floor
- player distance to ceiling
- next block x distance to player
- next blocks top y location
- next blocks bottom y location

The action space(2) :

- Up (press accelerator)
- Do nothing (don't press accelerator)

The reward function :

- For each vertical block it passes, it gains a positive reward of +1. Each time a terminal state is reached it receives a negative reward of -1.

Define the new Policy

- We need to have a deeper neural network since the environment is more complex

```
class Policy(nn.Module):
    def __init__(self, s_size, a_size, h_size):
        super(Policy, self).__init__()
        # Define the three layers here

    def forward(self, x):
        # Define the forward process here
        return F.softmax(x, dim=1)

    def act(self, state):
        state = torch.from_numpy(state).float().unsqueeze(0).to(device)
        probs = self.forward(state).cpu()
        m = Categorical(probs)
        action = m.sample()
        return action.item(), m.log_prob(action)
```

Solution

```
class Policy(nn.Module):
    def __init__(self, s_size, a_size, h_size):
        super(Policy, self).__init__()
        self.fc1 = nn.Linear(s_size, h_size)
        self.fc2 = nn.Linear(h_size, h_size * 2)
        self.fc3 = nn.Linear(h_size * 2, a_size)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return F.softmax(x, dim=1)

    def act(self, state):
        state = torch.from_numpy(state).float().unsqueeze(0).to(device)
        probs = self.forward(state).cpu()
        m = Categorical(probs)
        action = m.sample()
        return action.item(), m.log_prob(action)
```

Define the hyperparameters

- Because this environment is more complex.
- Especially for the hidden size, we need more neurons.

```
pixelcopter_hyperparameters = {
    "h_size": 64,
    "n_training_episodes": 50000,
    "n_evaluation_episodes": 10,
    "max_t": 10000,
    "gamma": 0.99,
    "lr": 1e-4,
    "env_id": env_id,
    "state_space": s_size,
    "action_space": a_size,
}
```

Train it

- We're now ready to train our agent .

```
# Create policy and place it to the device
# torch.manual_seed(50)
pixelcopter_policy = Policy(
    pixelcopter_hyperparameters["state_space"],
    pixelcopter_hyperparameters["action_space"],
    pixelcopter_hyperparameters["h_size"],
).to(device)
pixelcopter_optimizer = optim.Adam(pixelcopter_policy.parameters(), lr=pixelcopter_hyperparameters
    ["lr"])
```

```
scores = reinforce(
    pixelcopter_policy,
    pixelcopter_optimizer,
    pixelcopter_hyperparameters["n_training_episodes"],
    pixelcopter_hyperparameters["max_t"],
    pixelcopter_hyperparameters["gamma"],
```

```
    1000,
)
```

Publish our trained model on the Hub

```
repo_id = ""  # TODO Define your repo id {username/Reinforce-{model-id}}
push_to_hub(
    repo_id,
    pixelcopter_policy,  # The model we want to save
    pixelcopter_hyperparameters,  # Hyperparameters
    eval_env,  # Evaluation environment
    video_fps=30
)
```

Some additional challenges

The best way to learn **is to try things on your own**! As you saw, the current agent is not doing great. As a first suggestion, you can train for more steps. But also try to find better parameters.

In the Leaderboard you will find your agents. Can you get to the top?

Here are some ideas to climb up the leaderboard:

- Train more steps
- Try different hyperparameters by looking at what your classmates have done https://huggingface.co/models?other=reinforce
- **Push your new trained model** on the Hub
- **Improving the implementation for more complex environments** (for instance, what about changing the network to a Convolutional Neural Network to handle frames as observation)?

Congrats on finishing this unit! There was a lot of information. And congrats on finishing the tutorial. You've just coded your first Deep Reinforcement Learning agent from scratch using PyTorch and shared it on the Hub .

Don't hesitate to iterate on this unit **by improving the implementation for more complex environments** (for instance, what about changing the network to a Convolutional Neural Network to handle frames as observation)?

In the next unit, **we're going to learn more about Unity MLAgents**, by training agents in Unity environments. This way, you will be ready to participate in the **AI vs AI challenges where you'll train your agents to compete against other agents in a snowball fight and a soccer game.**

Sound fun? See you next time!

Finally, we would love **to hear what you think of the course and how we can improve it**. If you have some feedback then please fill this form

See you in Unit 5!

Keep Learning, stay awesome

7 Quiz

The best way to learn and to avoid the illusion of competence **is to test yourself.** This will help you to find **where you need to reinforce your knowledge**.

Q1: What are the advantages of policy-gradient over value-based methods? (Check all that apply)

- **A.** Policy-gradient methods can learn a stochastic policy
- **B.** Policy-gradient methods are more effective in high-dimensional action spaces and continuous actions spaces
- **C.** Policy-gradient converges most of the time on a global maximum.

Q2: What is the Policy Gradient Theorem?

Q3: What's the difference between policy-based methods and policy-gradient methods? (Check all that apply)

- **A.** Policy-based methods are a subset of policy-gradient methods.
- **B.** Policy-gradient methods are a subset of policy-based methods.
- **C.** In Policy-based methods, we can optimize the parameter θ **indirectly** by maximizing the local approximation of the objective function with techniques like hill climbing, simulated annealing, or evolution strategies.
- **D.** In Policy-gradient methods, we optimize the parameter θ **directly** by performing the gradient ascent on the performance of the objective function.

Q4: Why do we use gradient ascent instead of gradient descent to optimize J(θ)?

- **A.** We want to minimize J(θ) and gradient ascent gives us the direction of the steepest increase of J(θ)
- **B.** We want to maximize J(θ) and gradient ascent gives us the direction of the steepest increase of J(θ)

Congrats on finishing this Quiz , if you missed some elements, take time to read the chapter again to reinforce () your knowledge.

7.1 Quiz solutions and explanations

Q1 solution

Correct answer(s): A, B

A. Policy-gradient methods can learn a stochastic policy

B. Policy-gradient methods are more effective in high-dimensional action spaces and continuous actions spaces

C. Policy-gradient converges most of the time on a global maximum. Explanation: No, frequently, policy-gradient converges on a local maximum instead of a global optimum.

Q2 solution

The Policy Gradient Theorem is a formula that will help us to reformulate the objective function into a differentiable function that does not involve the differentiation of the state distribution.

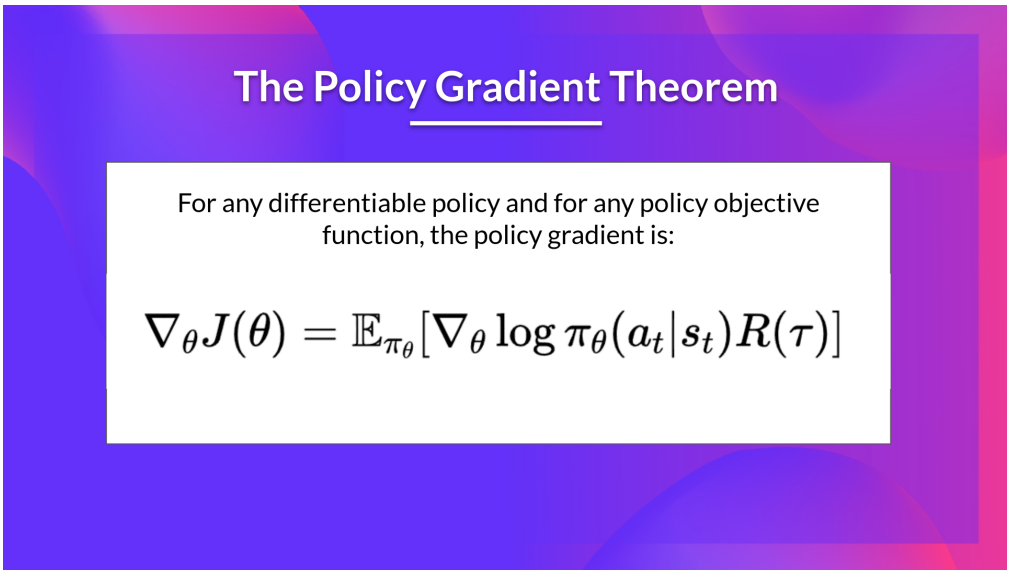


Figure 32: The policy-gradient theorem [original figure]

Q3 solution

Correct answer(s): B, C, D

A. Policy-based methods are a subset of policy-gradient methods.

B. Policy-gradient methods are a subset of policy-based methods.

C. In Policy-based methods, we can optimize the parameter θ **indirectly** by maximizing the local approximation of the objective function with techniques like hill climbing, simulated annealing, or evolution strategies.

D. In Policy-gradient methods, we optimize the parameter θ **directly** by performing the gradient ascent on the performance of the objective function.

Q4 solution

Correct answer(s): B

A. We want to minimize J(θ) and gradient ascent gives us the direction of the steepest increase of J(θ)

B. We want to maximize J(θ) and gradient ascent gives us the direction of the steepest increase of J(θ)

8 Conclusion

Congrats on finishing this unit! There was a lot of information. And congrats on finishing the tutorial. You've just coded your first Deep Reinforcement Learning agent from scratch using PyTorch and shared it on the Hub .

Don't hesitate to iterate on this unit **by improving the implementation for more complex environments** (for instance, what about changing the network to a Convolutional Neural Network to handle frames as observation)?

In the next unit, **we're going to learn more about Unity MLAgents**, by training agents in Unity environments. This way, you will be ready to participate in the **AI vs AI challenges where you'll train your agents to compete against other agents in a snowball fight and a soccer game.**

Sound fun? See you next time!

Finally, we would love **to hear what you think of the course and how we can improve it**. If you have some feedback then please fill this form

Keep Learning, stay awesome

9 Glossary

This is a community-created glossary. Contributions are welcome!

- **Deep Q-Learning:** A value-based deep reinforcement learning algorithm that uses a deep neural network to approximate Q-values for actions in a given state. The goal of Deep Q-learning is to find the optimal policy that maximizes the expected cumulative reward by learning the action-values.
- **Value-based methods:** Reinforcement Learning methods that estimate a value function as an intermediate step towards finding an optimal policy.
- **Policy-based methods:** Reinforcement Learning methods that directly learn to approximate the optimal policy without learning a value function. In practice they output a probability distribution over actions.

  The benefits of using policy-gradient methods over value-based methods include:

  - simplicity of integration: no need to store action values;
  - ability to learn a stochastic policy: the agent explores the state space without always taking the same trajectory, and avoids the problem of perceptual aliasing;
  - effectiveness in high-dimensional and continuous action spaces; and
  - improved convergence properties.

- **Policy Gradient:** A subset of policy-based methods where the objective is to maximize the performance of a parameterized policy using gradient ascent. The goal of a policy-gradient is to control the probability distribution of actions by tuning the policy such that good actions (that maximize the return) are sampled more frequently in the future.
- **Monte Carlo Reinforce:** A policy-gradient algorithm that uses an estimated return from an entire episode to update the policy parameter.

If you want to improve the course, you can open a Pull Request.

This glossary was made possible thanks to:

- Diego Carpintero

10 Additional Readings

These are **optional readings** if you want to go deeper.

10.1 Introduction to Policy Optimization

- Part 3: Intro to Policy Optimization - Spinning Up documentation

10.2 Policy Gradient

- https://johnwlambert.github.io/policy-gradients/
- RL - Policy Gradient Explained
- Chapter 13, Policy Gradient Methods; Reinforcement Learning, an introduction by Richard Sutton and Andrew G. Barto

10.3 Implementation

- PyTorch Reinforce implementation
- Implementations from DDPG to PPO